

SCUBEX: HERRAMIENTA Y RED SOCIAL PARA BUCEADORES

IVÁN FERNÁNDEZ LIMÁRQUEZ

Trabajo fin de Grado

Supervisado por Dr. Pablo Trinidad Martín-Arroyo



Universidad de Sevilla

mayo 2026

Publicado en mayo 2026 por

Iván Fernández Limárquez

Copyright © MMXXVI

<http://www.lsi.us.es/~trinidad>

ivaferlim@alum.us.es

Pon aquí cuestiones acerca del copyright

Yo, D. Iván Fernández Limárquez con NIF número 77015962W,

DECLARO

mi autoría del trabajo que se presenta en la memoria de este trabajo fin de grado que tiene por título:

Scubex: Herramienta y Red social para Buceadores

Lo cual firmo,

Fdo. D. Iván Fernández Limárquez
en la Universidad de Sevilla
05/05/2026

Para mi familia, amigos y para el mar que ha inspirado este proyecto.



AGRADECIMIENTOS

Agradecer a mi familia, amigos y conocidos que han probado mi aplicación. Con los que pese a haber sido tan pesado, han tenido la paciencia de seguir mis interesantísimas discusiones sobre temas como: Que color es mas fácil de ver, donde es objetivamente mas intuitivo que vaya un botón o, por vigesimotercera vez, escucharme sobre lo chula que esta la animación de inicio.

Especial mención a mi madre, quien sabe cuanto amo el mar y me ha impulsado y apoyado en todo momento.



RESUMEN

Scubex es una red social y herramienta pensada para buceadores que necesiten conocer diversos aspectos relacionados con las zonas de inmersión. La aplicación permite ver las especies en la zona y datos sobre ellas, condiciones climatológicas y compartir experiencias sobre la misma. Mediante publicaciones los usuarios podrán conectar unos con otros y conocer de la mano de demás buceadores si esa zona es interesante o no.

ÍNDICE GENERAL

I	Introducción	1
1.	Contexto	3
1.1.	Redes sociales y exploración marina	4
1.2.	Aplicaciones y tecnologías similares	4
2.	Objetivos	7
2.1.	Motivación	8
2.2.	Listado de objetivos	8
II	Organización del proyecto	11
3.	Metodología	13
3.1.	Estructura organizacional del proyecto	14
3.2.	Metodología de desarrollo	14
3.2.1.	Justificación de Feature-Driven Development (FDD)	14
3.2.2.	Adaptación del ciclo FDD para Scubex	15
3.2.3.	Herramientas de desarrollo	15
3.3.	Seguimiento y control	16
4.	Planificación	17

4.1. Resumen temporal del proyecto	18
4.2. Planificación inicial	18
5. Costes	19
5.1. Resumen de costes del proyecto	20
5.2. Costes de personal	20
5.2.1. Cálculo proporcional al tiempo dedicado	20
5.2.2. Costes sociales	20
5.2.3. Total costes de personal	21
5.3. Costes materiales	21
5.3.1. Amortización de equipamiento informático	21
5.3.2. Despliegue en la nube	21
5.3.3. Total costes materiales	22
5.4. Costes indirectos	22
5.5. Coste total del proyecto	22
 III Desarrollo del proyecto	 25
6. Arranque	27
6.1. Lista de características	28
6.2. Diseño arquitectónico	29
6.2.1. Stack Tecnológico	29
6.2.2. Estrategia de entornos	30
6.2.3. Justificación de MapLibre GL JS + MapTiler	31
6.2.4. Integración de APIs Externas	31
6.2.5. Diagrama de despliegue	33

7. Iteración 1: Inicio del MVP de Scubex	35
7.1. Características desarrolladas esta iteración	36
7.2. Diseño	40
7.3. Implementación	41
7.4. Despliegue	48
7.5. Pruebas	48
8. Iteración 2: Sistema Climático	51
8.1. Características desarrolladas esta iteración	52
8.2. Diseño	54
8.3. Implementación	54
8.4. Despliegue	59
8.5. Pruebas	60
9. Iteración 3: Publicaciones	61
9.1. Características desarrolladas esta iteración	62
9.2. Diseño	64
9.3. Implementación	65
9.4. Despliegue	70
9.5. Pruebas	70
10. Iteración 4: Interacciones y Red Social	73
10.1. Características desarrolladas esta iteración	74
10.2. Diseño	78
10.3. Implementación	78
10.4. Despliegue	84

10.5. Pruebas	84
IV Cierre del proyecto	87
11. Manual de usuario	89
11.1. Sección libre	90
12. Conclusiones	91
12.1. Informe post-mortem	92
12.1.1. Lo que ha ido bien	92
12.1.2. Lo que ha ido mal	92
12.1.3. Discusión	92
12.2. Trabajos futuros	92
Referencias bibliográficas	93

ÍNDICE DE FIGURAS

6.1. Diagrama UML de despliegue de Scubex	33
7.1. Diagrama UML de diseño para la iteración 1	40
8.1. Diagrama UML de diseño para la iteración 2	55
9.1. Diagrama UML de diseño para la iteración 3	64
10.1. Diagrama UML de diseño para la iteración 4	78

ÍNDICE DE CUADROS

4.1. Tabla resumen de tiempos y planificación	18
4.2. Planificación temporal de iteraciones	18
5.1. Tabla resumen de costes del proyecto	20
5.2. Coste total del proyecto Scubex	23
6.1. Estrategia de entornos del proyecto	30
6.2. Resumen de APIs externas	32
7.1. Análisis de valor aportado: Mapa Interactivo	37
7.2. Análisis de valor aportado: Escáner	38
7.3. Análisis de valor aportado: Autenticación	39
7.4. Memorando técnico 001: Geometría WKT	42
7.5. Memorando técnico 002: Filtrado de Calidad Visual	43
7.6. Memorando técnico 003: Caché L1 CachedScan	44
7.7. Memorando técnico 004: Caché L2: SpeciesEnrichmentCache	45
7.8. Memorando técnico 005: Autenticación OAuth2 + JWT	46
8.1. Análisis de valor aportado: Panel de Clima	53
8.2. Memorando técnico 006: WeatherResponse	56
8.3. Memorando técnico 007: Evaluación de Condiciones	57
8.4. Memorando técnico 008: Caché de Clima	58

9.1. Análisis de valor aportado: Publicaciones Geolocalizadas	63
9.2. Memorando técnico 009: Filtrado por Viewport del Mapa	66
9.3. Memorando técnico 010: Geocoding Inverso con Nominatim	67
9.4. Memorando técnico 011: Almacenamiento de Imágenes como BLOB . .	68
10.1. Análisis de valor aportado: Interacciones con Publicaciones	75
10.2. Análisis de valor aportado: Seguimiento de Usuarios y Perfiles Públicos	76
10.3. Análisis de valor aportado: Centro de Notificaciones	77
10.4. Memorando técnico 012: Likes, Guardados y Comentarios	80
10.5. Memorando técnico 013: Notificaciones y Menciones	81
10.6. Memorando técnico 014: Centro de Notificaciones	82

LISTA DE TAREAS PENDIENTES

PARTE I

INTRODUCCIÓN

CONTEXTO

The sea, once it casts its spell, holds one in its net of wonder forever.

*Jacques Yves Cousteau,
Oceanographer*

Vamos a dar una visión general del contexto en el que se desarrolla el proyecto, incluyendo el estado actual de la industria y el porqué del mismo.

1.1 REDES SOCIALES Y EXPLORACIÓN MARINA

Desde pequeño he vivido rodeado de mar, pero es solo ahora cuando he descubierto lo mucho que me gusta el buceo. Sin embargo, este viene con una serie de retos y problemas: ¿Dónde me sumerjo?, ¿Hará buen clima debajo del agua?, ¿Podré llegar a la zona con mi equipo o hará mucho viento? y, si voy, ¿Qué especies podré ver?

Con este proyecto busco dar una solución a estos problemas desde dos perspectivas. Estadística y personal, el usuario podrá basarse en cientos de datos ya recopilados de una zona, tomado de una base de datos que lleva creada y actualizandose desde el 1970. O simplemente podrá basarse en la experiencia de otros buceadores en esa misma zona.

1.2 APLICACIONES Y TECNOLOGÍAS SIMILARES

No tendría sentido desarrollar un proyecto que ya existe, por ello he buscado aplicaciones que hagan algo similar a la propuesta de Scubex. Que básicamente se basa en dar respuesta a las preguntas preguntas que he mencionado, para ello hay que dar información de: Que condiciones climatológicas hay en una zona, que especies marinas podrás ver y como está el mar ese día concreto en esa zona específica. Esto debe hacerse usando tanto datos recopilados a lo largo de los años de una zona, como dejando a los usuarios compartir sus experiencias personales. Buscando aplicaciones que den respuesta a estas preguntas he encontrado tres:

La primera es **Dive Mate**: Una aplicación que te muestra especies que has visto tú en una zona y que te da estimaciones climatológicas precisas. Sin embargo, la interfaz es tosca. Y no es posible compartir experiencias personales, un usuario no puede interactuar con otros ni registrar sus propias observaciones.

La segunda es **iNaturalist**: Una aplicación que te permite ver, en un mapa, avistamientos de especies en esa zona. Sus fotos e información de avistamientos son increíblemente ricas y útiles. Sin embargo, no está centralizada en buceadores y no incluye información climatológica para el mar de la zona.

La última es **DiveApp**: Una aplicación orientada a los avistamientos y preparación de viajes que incluye componentes de red social. Tiene el problema contrario a Dive Mate, incluye experiencia personal a modo de publicaciones y clima sobre la zona. Pero sin embargo no incluye datos ya recopilados sobre la zona en la que pretendes

sumergirte.

Como conclusión, hay aplicaciones que ofrecen funcionalidades parecidas a las que esta aplicación proporciona, sin embargo, ninguna las centraliza. Ni están implementadas de una forma intuitiva, ni con tecnologías modernas. En este hueco de mercado encaja Scubex. Por ello me he decidido a hacer este proyecto

OBJETIVOS

If you don't know where you are going, you'll end up someplace else.

*Yogi Berra,
Baseball player*

Visto el contexto, se formalizan los objetivos de la app de forma concreta. ¿Hacia quien esta dirigida la aplicación? ¿Qué funcionalidades tendrá? ¿Qué tecnologías se van a usar? ¿Qué se va a conseguir con el proyecto?

2.1 MOTIVACIÓN

Como se ha comentado, la idea de Scubex es centralizar funcionalidades para el nicho del buceo. Para lograrlo se han planteado esta serie de objetivos que, una vez cumplidos, darán como resultado la aplicación deseada.

2.2 LISTADO DE OBJETIVOS

Objetivo 1. Implementar un sistema de autenticación de usuarios

Desarrollar un sistema de autenticación seguro, permitiendo a los usuarios registrarse e iniciar sesión de forma sencilla. El sistema debe guardar los perfiles de usuario y datos tales como: nombre, foto y publicaciones.

Objetivo 2. Crear un mapa interactivo de exploración marina

Integrar un mapa interactivo con controles de zoom y navegación que permita a los usuarios explorar zonas de buceo de forma intuitiva y fluida.

Objetivo 3. Integrar datos científicos de biodiversidad marina

Desarrollar un escáner de especies que consulte APIs científicas para identificar fauna marina en una zona determinada.

Objetivo 4. Proporcionar información climatológica y oceanográfica en tiempo real

Ofrecer datos actualizados sobre condiciones climatológicas de la zona: viento, temperatura del agua, corrientes marinas, oleaje y demás datos atmosféricos relevantes para la planificación de inmersiones en una zona de inmersión.

Objetivo 5. Implementar un sistema de registro de publicaciones

Permitir a los usuarios publicar sus inmersiones en el mapa interactivo con título, descripción opcional, fotografía y coordenadas geográficas. El autor puede editar y eliminar sus propias publicaciones, y cada publicación es compartible mediante enlace directo.

Objetivo 6. Crear un sistema de interacción social sobre publicaciones

Desarrollar un sistema de interacciones sobre publicaciones: likes, guardado para acceso posterior y comentarios con menciones a otros usuarios. Las menciones generan notificaciones al usuario mencionado.

Objetivo 7. Implementar una red social de seguimiento entre usuarios

Desarrollar perfiles públicos navegables con número de seguidores y seguidos,

posibilidad de seguir o dejar de seguir a cualquier usuario, y acceso al historial de publicaciones de cada perfil. Los perfiles son compartibles mediante enlace directo.

Objetivo 8. Implementar un sistema de notificaciones

Proporcionar notificaciones al usuario sobre eventos relevantes: nuevos seguidores, likes, comentarios en sus publicaciones y menciones. Las notificaciones son gestionables, pudiendo borrarlas o marcarlas como leídas.

PARTE II

ORGANIZACIÓN DEL PROYECTO

METODOLOGÍA

Don't reinvent the wheel, just realign it.

*Anthony J. D'Angelo,
Escritor motivacional*

***E**ste capítulo describe el marco metodológico adoptado para el desarrollo de Scubex, basado en Feature-Driven Development (FDD) con adaptaciones específicas para un proyecto académico individual.*

3.1 ESTRUCTURA ORGANIZACIONAL DEL PROYECTO

Scubex se desarrolla como Trabajo Fin de Grado de forma individual, asumiendo las siguientes responsabilidades:

- **Arquitecto de Software:** Diseño de la arquitectura cliente-servidor y selección del stack tecnológico.
- **Desarrollador Full-Stack:** Implementación del backend (Spring Boot) y frontend (React + Vite).
- **Documentador Técnico:** Redacción de la memoria académica y especificación de APIs mediante OpenAPI (Swagger).

El tutor del proyecto proporciona la supervisión técnica y académica, validando decisiones arquitectónicas y avances iterativos.

3.2 METODOLOGÍA DE DESARROLLO

3.2.1 Justificación de Feature-Driven Development (FDD)

Se adopta **Feature-Driven Development** como metodología central por sus características idóneas para proyectos de alcance medio con requisitos funcionales bien definidos:

- **Orientación a características:** Cada funcionalidad (escáner de especies, integración climática, red social) se desarrolla como una única unidad. Lo que significa que se realizará el código y hasta que no esté documentada y acabada, no se pasará a la siguiente.
- **Trazabilidad:** Al forzar la compleción completa de una característica hay una relación estrecha y directa entre requisitos, código y documentación. Esto hace que sea fácil trazar un camino claro del avance del proyecto.
- **Escalabilidad:** Gracias a este enfoque es relativamente añadir nuevas características en el futuro. Ya que simplemente hay que añadir una característica a la lista y tratarla como esa unidad que previamente hemos mencionado.

En resumen, esta metodología permite estructurar el desarrollo con una lista de características claras y priorizadas. Lo que ayuda a mantener una organización coherente y simple en un equipo pequeño o individual.

3.2.2 Adaptación del ciclo FDD para Scubex

El modelo clásico de FDD consta de 5 fases. Para este proyecto académico se adapta la secuencia, unificando la fase de diseño y añadiendo dos fases específicas: una de documentación y una de seguimiento:

1. **Desarrollar la lista de características:** Identificación y priorización de funcionalidades nucleares (Gestión de usuarios OAuth2, Escáner de especies, Exploración climática, Red social).
2. **Construir un modelo global:** Diseño de la arquitectura del sistema a partir de las características identificadas (capítulo de Arranque).
3. **Planificar por característica:** Estimación tecnológica, limitaciones y consecuente diseño del flujo de datos.
4. **Construir por característica:** Implementación incremental con validación mediante pruebas constantes.
5. **Documentar la característica:** Redacción académica de cada característica en LaTeX, incluyendo justificación de decisiones técnicas y lecciones aprendidas.
6. **Seguimiento y control:** Al término de cada iteración se compara el resultado con lo planificado: fechas de inicio y fin, horas dedicadas y características completadas. Las desviaciones quedan registradas en la sección de seguimiento de cada capítulo de iteración.

3.2.3 Herramientas de desarrollo

- **Control de versiones:** Git + GitHub (commits atómicos por característica).
- **Gestión de dependencias:** Maven (backend), npm (frontend).
- **Testing:** JUnit 5 + Mockito (backend), Jacoco para generar los reportes de cobertura.

- **Documentación de API:** Swagger/OpenAPI 3.0 (generación automática de contratos REST).
- **Compilación:** Vite (desarrollo frontend con HMR), Spring Boot Maven Plugin (empaquetado JAR).
- **Despliegue:** Vercel (frontend, despliegue automático por rama con previews) + Railway (backend Spring Boot y PostgreSQL, entornos de preproducción y producción independientes).

3.3 SEGUIMIENTO Y CONTROL

El progreso del proyecto se monitoriza mediante:

- **Commits semánticos:** Mensajes (casi siempre) estructurados (feat, fix, docs, refactor) para trazabilidad histórica.
- **Reuniones de seguimiento:** Sesiones cada tres semanas con el tutor para validar decisiones arquitectónicas y resolver bloqueos técnicos.
- **Documentación incremental:** Redacción de capítulos inmediatamente tras completar cada característica dentro de la aplicación.

Métricas de avance:

- Características completadas.
- Cobertura de tests unitarios (objetivo: >70% en lógica de negocio).

PLANIFICACIÓN

Divide y vencerás · El arte de la guerra

*Sun Tzu (544–496 a.C.),
Maestro de guerra y escritor*

*E*n este capítulo se exponen las divisiones en sprint planificadas para la realización del proyecto.

4.1 RESUMEN TEMPORAL DEL PROYECTO

Resumen del proyecto	
Fecha de inicio	13/10/2025
Fecha de fin	XX/XX/2026
Periodicidad de las revisiones	3 semanas
Carga de trabajo semanal	12 horas
Horas totales previstas	285 horas
Horas finales	293 horas

Cuadro 4.1: Tabla resumen de tiempos y planificación

4.2 PLANIFICACIÓN INICIAL

Aquí un desglose de las iteraciones, comienzo y fin de cada una:

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 08/05/26

Cuadro 4.2: Planificación temporal de iteraciones

La primera iteración cubre un período más largo que las siguientes: el proyecto arranca en octubre de 2025, pero la documentación no comienza hasta febrero de 2026. El trabajo de implementación realizado durante ese período no se descarta, sino que se incorpora en esta primera iteración. Las iteraciones siguientes se planifican con una duración de tres semanas cada una, lo que permite mantener un ritmo constante de trabajo y dedicar revisiones periódicas con el tutor al progreso del proyecto.

COSTES

Here Comes the Money

*Jim Johnston y Naughty by Nature,
Escritores y Raperos*

***E**n este capítulo se realiza el desglose de los costes asociados al proyecto, basándose en los salarios reales de puestos tecnológicos.*

5.1 RESUMEN DE COSTES DEL PROYECTO

Resumen del proyecto	
Costes de personal	5.290,28 €
Desarrollador Full-Stack Junior	5.290,28 €
Costes materiales	67,23 €
Costes indirectos	535,75 €
TOTAL	5.893,26 €

Cuadro 5.1: Tabla resumen de costes del proyecto

5.2 COSTES DE PERSONAL

Para el cálculo de los costes de personal se han consultado los datos de la **Guía Salarial 2026** de Manfred¹, que recopila información salarial de más de 120.000 profesionales de tecnología en España.

El desarrollo de Scubex ha sido asumido íntegramente por un único perfil: un **Desarrollador Full-Stack Junior** con menos de dos años de experiencia, responsable tanto de la implementación como de la documentación. Según la guía salarial, este perfil se sitúa en el rango de 20.000–30.000€ anuales; se ha tomado el valor medio de **25.000€ brutos anuales**.

5.2.1 Cálculo proporcional al tiempo dedicado

La carga de trabajo estimada para el TFG es de **293 horas**, frente a una jornada laboral anual estándar de **1.800 horas**. El coste proporcional se calcula como:

$$\text{Salario proporcional} = 25,000\text{€} \times \frac{293}{1,800} = 4,069,44\text{€} \quad (5.1)$$

5.2.2 Costes sociales

A este salario bruto se le añaden los **costes sociales** (Seguridad Social, contingencias comunes, desempleo, formación profesional, etc.), que suponen aproximadamente el

¹<https://www.getmanfred.com/blog/guia-salarial-2026-salarios-en-tecnologia-espana-manfred>

30 % del salario bruto:

$$\text{Costes sociales} = 4,069,44\text{€} \times 0,30 = 1,220,83\text{€} \quad (5.2)$$

5.2.3 Total costes de personal

$$\text{Total} = 4,069,44\text{€} + 1,220,83\text{€} = 5,290,28\text{€} \quad (5.3)$$

5.3 COSTES MATERIALES

5.3.1 Amortización de equipamiento informático

Según el criterio de amortización fiscal establecido por la Agencia Tributaria española, los equipos informáticos se amortizan a un **máximo del 25 % anual** (vida útil de 4 años).

Para el desarrollo del proyecto se ha utilizado un portátil con un coste de adquisición de **1.200€**:

- **Amortización anual:** $1,200\text{€} \times 0,25 = 300\text{€}$
- **Amortización proporcional** (293h / 1.800h): $300\text{€} \times \frac{293}{1,800} = 48,83\text{€}$

5.3.2 Despliegue en la nube

El proyecto se encuentra desplegado en producción y preproducción mediante dos servicios:

- **Railway** (backend Spring Boot + PostgreSQL): Plan Hobby a 5 USD/mes (~4,60 €/mes). Este plan incluye 5USD de créditos de uso mensuales; superado ese umbral, se factura por consumo real de recursos. Actualmente debido al tráfico bajo-moderado, el crédito mensual de 5 USD resulta suficiente.
- **Vercel** (frontend React/Vite): Plan Hobby gratuito, sin coste.

Coste estimado de despliegue durante el desarrollo del TFG (4 meses):

$$\text{Railway} = 4,60 \text{ €/mes} \times 4 \text{ meses} = 18,40 \text{ €} \quad (5.4)$$

$$\text{Total despliegue} = 18,40 \text{ €} + 0 \text{ € (Vercel)} = 18,40 \text{ €} \quad (5.5)$$

En un escenario de producción real con mayor tráfico, los costes de Railway podrían aumentar proporcionalmente al uso de CPU, RAM y almacenamiento consumidos más allá del crédito incluido. En este caso podría crearse un plan personalizado, que incluya las necesidades concretas de la aplicación.

5.3.3 Total costes materiales

$$\text{Total} = 48,83 \text{ €} + 18,40 \text{ €} = 67,23 \text{ €} \quad (5.6)$$

5.4 COSTES INDIRECTOS

Los costes indirectos incluyen gastos generales asociados al desarrollo del proyecto que no se imputan directamente: electricidad, conexión a internet, espacio de trabajo, cuotas mensuales de asistentes de IA, etc.

Se ha tomado un valor medio del **10%** de la suma de costes de personal y materiales:

$$\text{Costes indirectos} = (5,290,28 \text{ €} + 67,23 \text{ €}) \times 0,10 = 535,75 \text{ €} \quad (5.7)$$

5.5 COSTE TOTAL DEL PROYECTO

Desglose de costes	
Concepto	Importe
Costes de personal	5.290,28€
Desarrollador Full-Stack Junior	5.290,28€
Costes materiales	67,23€
Amortización equipamiento	48,83€
Despliegue Railway (4 meses)	18,40€
Costes indirectos	535,75€
TOTAL PROYECTO	5.893,26€

Cuadro 5.2: Coste total del proyecto Scubex

PARTE III

DESARROLLO DEL PROYECTO

ARRANQUE

The best way to get started is to quit talking and begin doing.

Walt Disney (1901–1966),

Empresario y cineasta

***E**ste capítulo define la lista inicial de características priorizadas del proyecto Scubex y, a partir de ellas, la arquitectura base necesaria para soportarlas. Se sigue el orden natural del ciclo FDD: primero se establece qué se va a construir, y solo entonces se diseña cómo.*

6.1 LISTA DE CARACTERÍSTICAS

Siguiendo la fase 1 de FDD (Desarrollar lista de características) y los objetivos definidos para el proyecto, se han identificado las siguientes funcionalidades nucleares priorizadas por valor para el usuario:

1. **Gestión de Usuarios (OAuth2):** Autenticación delegada mediante Google y persistencia de perfil (nombre, foto). El usuario puede personalizar su nombre y foto de perfil desde la página de perfil, con soporte para eliminar la cuenta.
2. **Mapa Interactivo:** Visualización de un mapa navegable mediante MapLibre GL JS como superficie central de interacción, con búsqueda de lugares integrada (Nominatim).
3. **Escáner de Especies:** Identificación de fauna marina en una zona mediante radio de búsqueda y validación cruzada de APIs científicas (OBIS + iNaturalist), con sistema de caché adaptativo (TTL 48h) para reducir latencia y llamadas externas.
4. **Datos Climáticos y Oceanográficos:** Condiciones de la zona bajo demanda mediante Open-Meteo (viento, temperatura del agua, corrientes, olas, datos atmosféricos), con evaluación de aptitud para el buceo y caché de 30 minutos en el backend.
5. **Registro de Publicaciones:** Publicación de inmersiones en el mapa con título, descripción, fotografía y geolocalización inversa (nombre del lugar mediante Nominatim). Edición y borrado por el propio autor. Cada publicación genera un enlace compartible (`/map?pub={id}`).
6. **Interacciones Sociales:** Sistema de likes, guardados y comentarios sobre publicaciones. Soporte de menciones en comentarios mediante la sintaxis `@[Nombre] (email)` con notificaciones automáticas al mencionado. El enlace compartible de cada publicación es accesible mediante un botón que usa la Web Share API en móvil y copia al portapapeles en escritorio.
7. **Red Social y Seguimiento de Usuarios:** Perfiles públicos por usuario con recuento de seguidores y seguidos, posibilidad de seguir/dejar de seguir, y vista de publicaciones de cada perfil. Página de perfil propio con mis publicaciones y publicaciones guardadas. Cada perfil dispone de enlace compartible (`/user/{email}`) accesible desde un botón de compartir en la página de perfil.

8. **Sistema de Notificaciones:** Notificaciones en tiempo real (campana en el header) para eventos de seguimiento, like, comentario y mención. Las notificaciones se marcan como leídas al abrirlas o se eliminan permanentemente con swipe (móvil) o botón (escritorio).

Justificación de priorización: La lista anterior refleja el orden de implementación planificado, de mayor a menor prioridad. A continuación se justifica ese orden:

- La autenticación se implementa primero ya que el resto de funcionalidades (avistamientos, perfil, red social) dependen de la identidad del usuario.
- El mapa es la superficie central de la aplicación: sin él no tiene sentido geolocalizar especies, condiciones climáticas ni avistamientos.
- El escáner de especies, los datos climáticos y el registro de publicaciones constituyen el MVP funcional mínimo. Como se planteó en el capítulo de Contexto, el objetivo es que el usuario pueda tomar decisiones apoyándose tanto en datos objetivos como en experiencias reales: el escáner y el clima aportan la primera dimensión; las publicaciones, la segunda, al permitir consultar las inmersiones documentadas por otros buceadores en la misma zona.
- Las funcionalidades de comunidad (likes, comentarios, seguimiento entre usuarios, notificaciones) se implementan sobre ese MVP: convierten la herramienta en una red social de buceadores, pero no son necesarias para que la propuesta de valor principal sea utilizable.

6.2 DISEÑO ARQUITECTÓNICO

Con las características identificadas, el sistema requiere una arquitectura capaz de orquestar múltiples APIs externas, gestionar identidad de usuario y servir una interfaz cartográfica interactiva. Se ha optado por una **arquitectura cliente-servidor desacoplada** con patrón Gateway de APIs.

6.2.1 Stack Tecnológico

- **Backend:** Spring Boot 4 (Java 21). Actúa como Gateway de APIs científicas, orquestador de peticiones asíncronas y gestor de autenticación.

- **Frontend:** React 18 + Vite. Con Hot Module Replacement (HMR) para desarrollo ágil. Gestión de estado reactiva mediante MobX `makeAutoObservable`, con stores separadas por dominio (especies, clima, publicaciones, usuario, mapa).
- **Cartografía:** MapLibre GL JS. Motor de renderizado vectorial para animaciones fluidas y bajo consumo de datos.
- **Base de Datos:** H2 en memoria para el entorno de desarrollo y tests unitarios. PostgreSQL 16 para preproducción y producción, gestionado como servicio en Railway.
- **Testing:** JUnit 5 + Mockito para pruebas unitarias del backend. Jacoco para generar reportes. H2 en memoria como base de datos de test.
- **Seguridad:** Spring Security con OAuth2 Client para Google Login.
- **Documentación de API:** OpenAPI 3.0 (Swagger UI) con anotaciones automáticas.
- **Despliegue:** Railway (backend + PostgreSQL) y Vercel (frontend).

6.2.2 Estrategia de entornos

Se han definido tres entornos que reflejan el ciclo de vida del software desde el desarrollo local hasta la puesta en producción:

Estrategia de entornos de Scubex			
Entorno	Infraestructura	Base de datos	Rama Git
Desarrollo	Local	H2 en memoria	Cualquiera
Preproducción	Railway + Vercel	PostgreSQL (Railway)	develop
Producción	Railway + Vercel	PostgreSQL (Railway)	main

Cuadro 6.1: Estrategia de entornos del proyecto

Justificación de la separación entre plataformas:

- **Vercel** gestiona el frontend React/Vite: Se despliega automáticamente en cada push. Se cuenta con previews por rama sin configuración adicional, más que establecer las direcciones de backend de producción y preproducción.
- **Railway** gestiona el backend Spring Boot y PostgreSQL en el mismo proyecto. Permite tener varios entornos distintos de una misma aplicación desplegados simultáneamente, lo que la hace idónea para combinarla con Vercel.
- La separación permite desplegar frontend y backend de forma independiente. Gracias a esto pueden escalarse independientemente y no solo eso. Debido a la separación se cuenta con mas memoria de los planes gratuitos de ambas aplicaciones, reduciendo costes.

La configuración CORS se ha establecido manualmente mediante la variable `CORS_ALLOWED_ORIGINS` en Railway, y `VITE_API_BASE_URL` en Vercel, configuradas con las respectivas URL del backend correspondientes a preproducción y producción.

6.2.3 Justificación de MapLibre GL JS + MapTiler

La solución cartográfica se compone de dos piezas complementarias:

- **MapLibre GL JS** actúa como motor de renderizado: biblioteca open source (fork de Mapbox GL JS) que renderiza estilos vectoriales mediante WebGL. Permite animaciones fluidas, rotación e inclinación del mapa, y menor consumo de datos frente a tiles rasterizados. Al ser open source, no impone restricciones de licencia sobre el código.
- **MapTiler** provee los tiles y el estilo visual del mapa a través de su API (`VITE_MAPTILER_KEY`). Se ha creado un estilo personalizado (tema marino) alojado en MapTiler Cloud. El plan gratuito cubre de sobra el volumen de peticiones necesario para el proyecto.

La separación entre motor de renderizado (MapLibre) y proveedor de tiles (MapTiler) permite sustituir el proveedor de tiles sin cambiar el código de renderizado, y viceversa.

6.2.4 Integración de APIs Externas

Estrategia de consumo:

APIs externas integradas en Scubex		
API	Proveedor	Datos proporcionados
OBIS	Ocean Biodiversity Information System	Ocurrencias biológicas: nombre científico, coordenadas, taxonomía (phylum), fecha de avistamiento.
iNaturalist	iNaturalist.org	Metadatos multimedia: fotos de especies, nombres comunes (preferred_common_name).
Open-Meteo	Open-Meteo.com	Datos climáticos y oceanográficos: viento, temperatura del agua/aire, altura de olas, corrientes marinas, precipitaciones. API gratuita sin necesidad de clave y con plan gratuito generoso.
Nominatim	OpenStreetMap Foundation	Geocodificación directa e inversa. Directa (/search): buscador de lugares del mapa. Inversa (/reverse): obtención del nombre humano de unas coordenadas (playa, municipio, estado) al crear publicaciones y al visualizar su detalle.

Cuadro 6.2: Resumen de APIs externas

- **OBIS + iNaturalist:** Consultas de especies marinas mediante el botón de escaneo.
- **Open-Meteo:** Consulta bajo demanda al interactuar con el mapa.
- **Nominatim:** Consulta bajo demanda al interactuar con el mapa. Se usa en el buscador de lugares (/search) y al crear una publicación para obtener el nombre humano de las coordenadas seleccionadas (/reverse).

Decisiones pendientes que quedaron en esta fase: Quedó pendiente el determinar la estrategia de caching para OBIS que se resolvió en la Iteración 1; Y también el caching de condiciones climáticas que se resolvió en la Iteración 2.

6.2.5 Diagrama de despliegue

La Fig. §6.1 muestra la topología de despliegue de Scubex: los nodos físicos o lógicos que ejecutan cada componente y los canales de comunicación entre ellos.

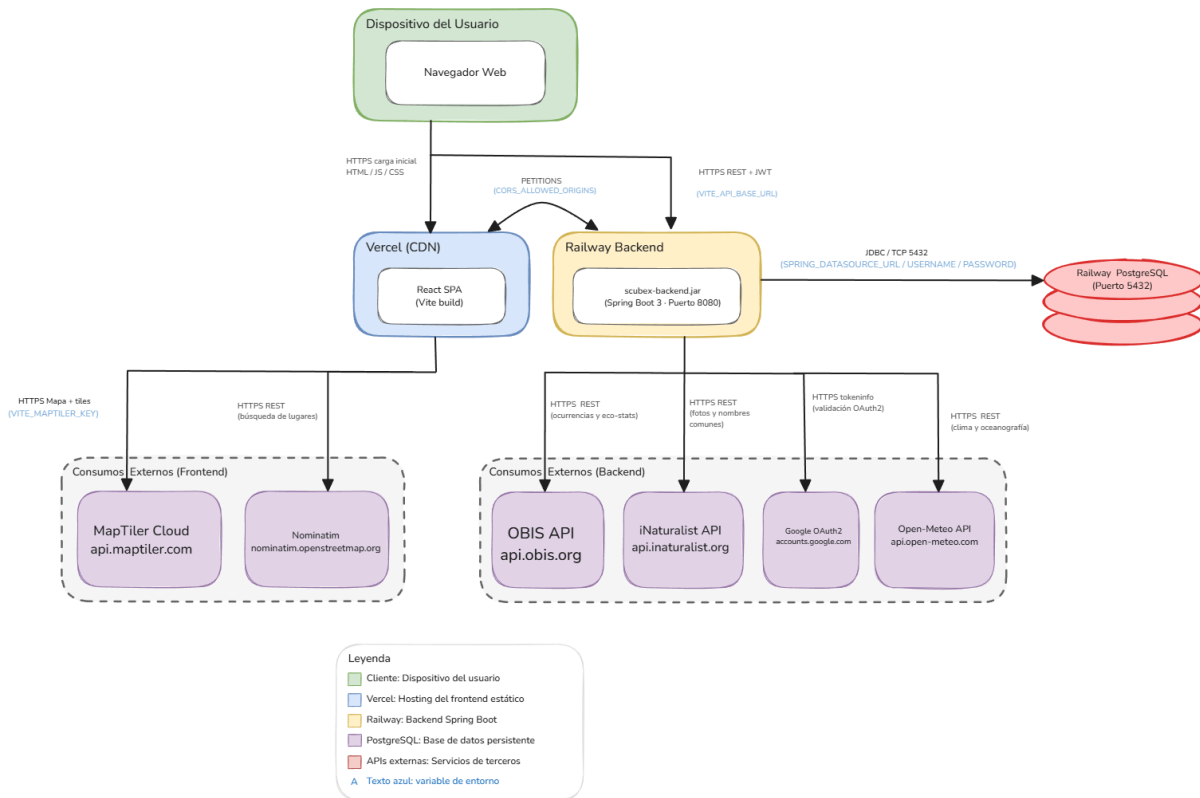


Figura 6.1: Diagrama UML de despliegue de Scubex

El navegador del usuario se comunica con dos nodos de producción: Vercel, que sirve la aplicación React compilada como artefacto estático mediante HTTPS, y el backend en Railway, al que envía las peticiones REST autenticadas con JWT. Las llamadas al backend deben pasar el filtro CORS configurado mediante `CORS_ALLOWED_ORIGINS`.

Los consumos externos se reparten en dos grupos según quién los origina. El frontend llama directamente a MapTiler Cloud para obtener los tiles y el estilo del mapa, y a Nominatim para la búsqueda y geocodificación inversa de lugares. El backend es el que se comunica con las APIs científicas y de datos: OBIS e iNaturalist para el escáner de biodiversidad, Open-Meteo para los datos climáticos y oceanográficos, y Google OAuth2 para validar los tokens de autenticación. El backend accede a PostgreSQL en

Railway mediante JDBC.

ITERACIÓN 1: INICIO DEL MVP DE SCUBEX

*E*sta iteración entrega parte del MVP funcional de Scubex: el mapa interactivo con escáner de biodiversidad marina y autenticación con Google.

7.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Mapa interactivo con capas de teselas, batimetría y búsqueda de lugares. Ver Tabla §7.1.
2. Escáner de biodiversidad marina con enriquecimiento OBIS + iNaturalist. Ver Tabla §7.2.
3. Autenticación con Google OAuth2 y emisión de JWT. Ver Tabla §7.3.

Esta iteración construye el núcleo de Scubex. El punto de partida es el mapa: sin una superficie interactiva donde el usuario pueda moverse y fijar zonas, ninguna otra funcionalidad tiene sentido. Sobre ese mapa se monta el escáner de biodiversidad, que es parte de la propuesta de valor diferencial de la aplicación. Y para que el usuario pueda guardar y compartir lo que encuentra, se añade la autenticación. Las tres características se analizan a continuación.

Con el mapa como superficie, lo siguiente es el escáner de biodiversidad: la funcionalidad que permite al buceador descubrir qué fauna marina habita la zona que está mirando (Tabla §7.2).

Por último, para que el usuario pueda identificarse y, en iteraciones futuras, publicar y guardar contenido, se necesita un sistema de autenticación. Se opta por delegar la identidad en Google para evitar gestionar contraseñas propias (Tabla §7.3).

Análisis de valor aportado: Mapa Interactivo	
Propuesta	Mapa interactivo con estilo personalizado y representación visual de batimetría. Incluye buscador de lugares y visualización en tiempo real de la zona de escaneo sobre el mapa.
Valor	Proporciona al usuario la interfaz principal de Scubex: localizar zonas de buceo sobre un mapa real con información de batimetría, fijar el centro con un click y visualizar dinámicamente el área de interés antes de lanzar el escáner.
Coste	Integración de la librería de mapas y uso de la API de MapTiler para estilos y batimetría propios. Conexión con Nominatim para la búsqueda de lugares. Requiere clave de API de MapTiler (plan gratuito: 5.000 sesiones/mes).
Opciones	Lista de spots predefinidos: el usuario elige de un catálogo fijo de zonas de buceo conocidas; más simple pero elimina la libertad de explorar cualquier punto. Buscador de texto: el usuario escribe el nombre del lugar y el sistema devuelve las especies de esa zona; se ha descartado por la falta de feedback visual. El mapa aporta orientación geográfica y espontaneidad que estas alternativas no ofrecen.
Riesgos	Límite mensual de sesiones del plan gratuito de MapTiler. Si el estilo base cambia o se elimina en MapTiler Cloud, la aplicación perdería el estilo del mapa. Dependencia de Nominatim (servicio público sin SLA).
Deuda técnica	La capa de batimetría proviene del estilo base de MapTiler; si cambia maptiler, la capa programática dejaría de funcionar y no se verían las diferentes profundidades.

Cuadro 7.1: Análisis de valor aportado: Mapa Interactivo

Análisis de valor aportado: Escáner de Biodiversidad	
Propuesta	Escáner de biodiversidad marina que consulta registros científicos de OBIS y enriquece cada especie con foto y nombre común de iNaturalist.
Valor	Permite descubrir la fauna marina real de una zona validada científicamente (>150M registros OBIS) con representación visual. Al desplegar la tarjeta de cada especie, el usuario puede consultar su rango de profundidad, temperatura, primer y último avistamiento, registros globales y categoría IUCN. La caché reduce la latencia de consultas repetidas sobre la misma zona.
Coste	Integración con dos APIs externas (OBIS e iNaturalist) y diseño de una caché de dos niveles para evitar latencias excesivas. Esfuerzo alto en la primera iteración por la coordinación de múltiples llamadas en paralelo.
Opciones	Buscador de especie: el usuario busca una especie concreta y la app le indica en qué zonas se encuentra; útil si ya sabes lo que buscas. Sin embargo el escáner es más intuitivo y expone todo el rango de especies de una zona sin que el usuario necesite saber de antemano qué buscar.
Riesgos	Rate limit de iNaturalist (100 req/min). Inconsistencias entre bases de datos (especie en OBIS sin entrada en iNaturalist). Latencia acumulada en primer escaneo de una zona sin caché.
Deuda técnica	Paginación pendiente (limitado a las primeras 1000 ocurrencias de OBIS). Ausencia de invalidación manual de caché si los datos cambian antes del TTL.

Cuadro 7.2: Análisis de valor aportado: Escáner

Análisis de valor aportado: Autenticación	
Propuesta	Inicio de sesión con cuenta de Google en un solo clic. El backend verifica la identidad con Google y emite un token propio que autentica las peticiones posteriores del usuario.
Valor	Elimina la fricción de registro manual. El usuario usa su identidad Google ya verificada. El JWT permite autenticar peticiones futuras sin nueva validación contra Google.
Coste	Configuración de credenciales en Google Cloud Console y gestión segura de los secretos de la aplicación. Implementación del filtro de seguridad que valida el token en cada petición.
Opciones	Auth0 / Firebase Auth: evita implementar la lógica JWT, pero añade dependencia de servicio externo y coste. Registro propio con email/contraseña: mayor fricción y necesidad de gestionar contraseñas.
Riesgos	Compromiso de JWT_SECRET: todos los tokens activos quedan expuestos. Expiración de 7 días sin mecanismo de refresco: el usuario debe volver a autenticarse al caducar el token.
Deuda técnica	Sin <i>refresh tokens</i> : al expirar el JWT el usuario debe volver a hacer login. Sin revocación de tokens comprometidos.

Cuadro 7.3: Análisis de valor aportado: Autenticación

7.2 DISEÑO

El diagrama de la Fig. §7.1 muestra cómo se organiza el código internamente. Los controladores reciben las peticiones del usuario y las delegan en los servicios, que contienen toda la lógica. Los repositorios son la capa que habla con la base de datos. Lo más destacable del diagrama es la presencia de dos entidades de caché: `CachedScan` y `SpeciesEnrichmentCache`. Que son el resultado directo de las decisiones de rendimiento que se explican en la sección de implementación.

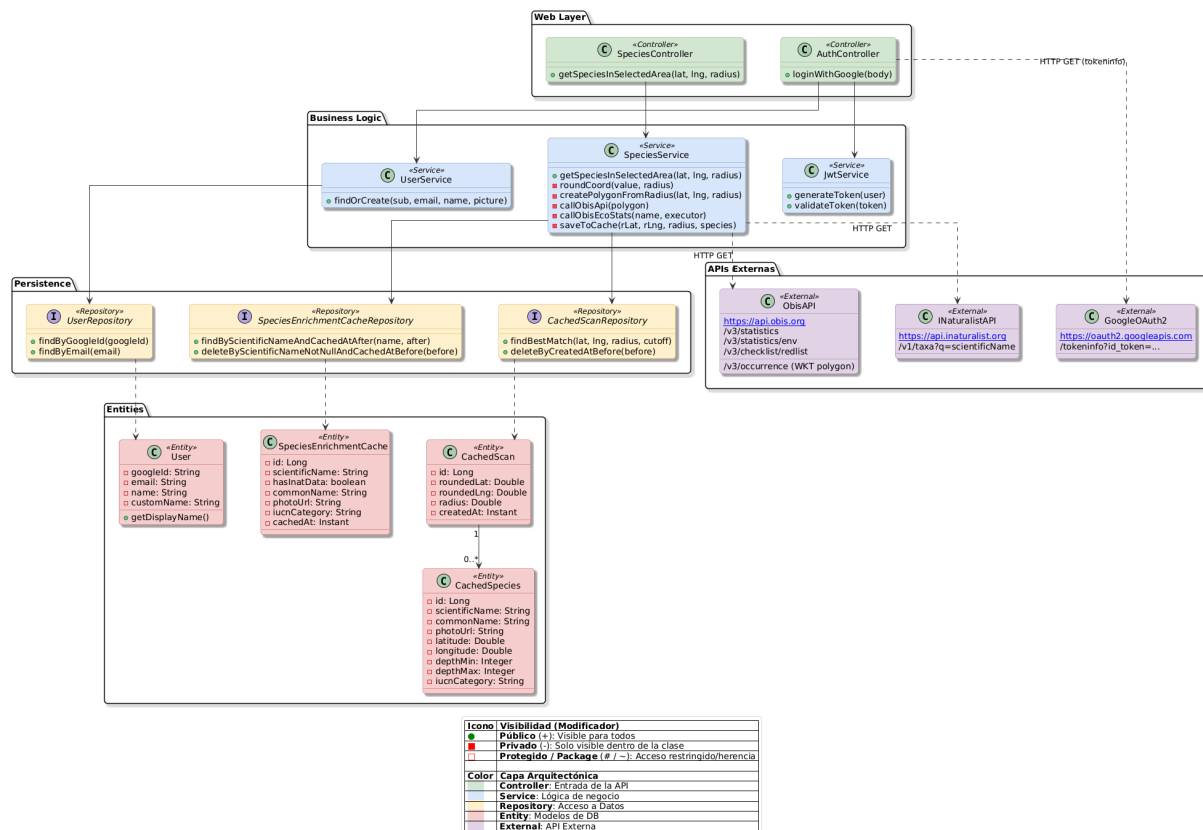


Figura 7.1: Diagrama UML de diseño para la iteración 1

La separación en dos niveles de caché uno por zona escaneada y principalmente por especie individual es la decisión que más impacto tiene sobre la experiencia de uso: sin ella, cada consulta al escáner tardaría entre 10 y 20 segundos. Con ella, las zonas responden de forma razonable y en algunos casos casi instantánea.

7.3 IMPLEMENTACIÓN

Para implementar el escáner se consultaron dos APIs externas (OBIS e iNaturalist) y se diseñó una caché de dos niveles para mantener los tiempos de respuesta aceptables. La autenticación se delegó en Google para evitar gestionar credenciales propias. Los cinco desafíos técnicos y las decisiones adoptadas para resolverlos están recogidos en las tablas siguientes:

- **Geometría de búsqueda** (Tabla §7.4): OBIS exige un polígono WKT, no un radio. Se genera uno de 12 vértices a partir del punto y radio elegidos por el usuario.
- **Filtrado de calidad** (Tabla §7.5): OBIS devuelve microorganismos sin valor visual. Se descartan las especies sin entrada en iNaturalist.
- **Caché de zona** (Tabla §7.6): un escaneo puede tardar 10–20 s. Se persiste el resultado por zona durante 48 h para evitar repetir las llamadas.
- **Caché de enriquecimiento** (Tabla §7.7): la misma especie en otra zona volvería a llamar a iNaturalist. Se guarda por nombre científico durante 30 días.
- **Autenticación OAuth2 + JWT** (Tabla §7.8): sin servidor de sesiones, la identidad viaja en cada petición. Se valida el token de Google y se emite un JWT propio de 7 días.

Memorando técnico 001: Geometría de Búsqueda WKT	
Asunto	La API de OBIS no acepta búsqueda radial: requiere un polígono POLYGON((. . .)) en formato WKT.
Resumen	Generación de un polígono de 12 vértices aproximadamente circular desde coordenadas centrales y radio.
Factores causantes	El usuario define la zona como un punto central y un radio, pero la API de OBIS exige el área en formato WKT (polígono cerrado). Alternativamente, se podría haber usado un bounding box rectangular, no obstante sobreestimaría el área incluyendo registros fuera del radio real.
Solución	Se implementa <code>createPolygonFromRadius(lat, lng, radius)</code> , que aproxima el círculo con un polígono de 12 vértices equidistantes, corrigiendo la longitud por $\cos(\phi)$ para no deformar el área en latitudes altas. La cadena WKT resultante se codifica adecuadamente antes de insertarse en la URL de la petición HTTP.
Motivación	OBIS alberga >150M registros de biodiversidad marina validados científicamente. Es preferible adaptarse a su interfaz antes que buscar fuentes alternativas de menor calidad, ya que no existen muchas comparables.
Cuestiones abiertas	A latitudes altas (> 60) la aproximación esférica introduce error; para el caso de uso de Scubex (zonas costeras tropicales y templadas) es despreciable.
Alternativas	Bounding box: sencillo pero incluye esquinas fuera del radio real. Filtrado post-proceso: pedir área global y filtrar en memoria, descartado por transferencia de datos innecesaria.

Cuadro 7.4: Memorando técnico 001: Geometría WKT

Memorando técnico 002: Filtrado de Calidad Visual	
Asunto	OBIS devuelve microorganismos y especies sin fotografía, que carecen de valor para una app de buceo.
Resumen	Filtrado cruzado: se descartan especies sin entrada en iNaturalist. Las que tienen entrada pero carecen de fotografía se incluyen con un emoji representativo de su phylum como fallback visual en el frontend.
Factores causantes	(1) OBIS incluye todo tipo de registros biológicos marinos (bacterias, algas unicelulares, etc.) que no se ven al bucear. (2) Hay especies macroscópicas con nombre científico válido que no tienen observaciones fotografiadas en iNaturalist (ej. <i>The-nea muricata</i>).
Solución	Cada especie devuelta por OBIS se busca en iNaturalist. Si no aparece, se descarta: sin nombre común ni foto no aporta valor. Si aparece, se incluye siempre: si tiene foto se usa; si no, el frontend muestra un emoji según el phylum como fallback visual.
Motivación	Un buceador busca animales que pueda ver en el agua. Incluir microorganismos sería ruido.
Cuestiones abiertas	La latencia de N llamadas HTTP a iNaturalist (una por especie única) en el primer escaneo de una zona puede superar los 20 segundos. Resuelta en Memorando 004 mediante caché de enriquecimiento por especie.
Alternativas	Placeholder genérico: asignar siempre un emoji o imagen por grupo taxonómico sin consultar iNaturalist. Elimina las llamadas externas pero pierde la foto real y el nombre común cuando sí existen.

Cuadro 7.5: Memorando técnico 002: Filtrado de Calidad Visual

Memorando técnico 003: Caché L1 de Escaneos (CachedScan)	
Asunto	Latencia de 10–20 s por escaneo en zonas sin datos previos en caché.
Resumen	Caché de resultado completo por zona geográfica indexado por coordenadas redondeadas y radio, con TTL de 48 h.
Factores causantes	Un escaneo completo puede tardar entre 10 y 20 segundos. Repetir ese proceso cada vez que alguien consulta la misma zona es un desperdicio innecesario, tanto en tiempo de respuesta como en llamadas a APIs externas.
Solución	El resultado de cada escaneo se persiste en base de datos bajo una clave compuesta por coordenadas redondeadas y radio. El redondeo agrupa clicks cercanos en la misma clave, evitando duplicados por diferencias de centímetros. Si existe un escaneo de menos de 48 h para esa zona, se devuelve directamente sin tocar ninguna API externa.
Motivación	Capacidad de dar una respuesta instantánea a un escaneo, sin pasar por APIs. Plugins ya preconfigurados en base de datos, lo que hace la implementación mas simple
Cuestiones abiertas	Clicks en el umbral de redondeo (ej. 0.049° vs 0.051°) pueden generar un falso miss aunque las zonas sean prácticamente idénticas. Comportamiento aceptado por la alta complejidad frente al bajo impacto que la resolución de este llevaba.
Alternativas	Redis: más rápido, pero añade servicio externo y coste. Spring Cache en memoria: no persiste entre reinicios del servidor (Railway reinicia en cada despliegue).

Cuadro 7.6: Memorando técnico 003: Caché L1 CachedScan

Memorando técnico 004: Caché L2 de Enriquecimiento: (SpeciesEnrichmentCache)	
Asunto	Aun con el caché L1, los escaneos seguían siendo lentos debido a los ratelimit de iNaturalist.
Resumen	Se implementó un caché persistente por especie (con scientificName como clave única), que guarda los datos extra de iNaturalist y básicos de OBIS durante 30 días para ahorrar llamadas. También se implementó un semáforo de concurrencia, para limitar las llamadas en paralelo a iNaturalist.
Factores causantes	Pese a que 100 llamadas por minuto sean usualmente suficientes, a veces se quedaban cortas en zonas de alta biodiversidad. La caché L1 evita repetir escaneos de la misma zona, pero no ayuda cuando la misma especie aparece en zonas distintas.
Solución	La foto, el nombre común y los datos ecológicos de cada especie se guardan en base de datos usando el nombre científico como clave, con una validez de 30 días. La próxima vez que esa especie aparezca en cualquier escaneo, se recupera directamente de base de datos sin tocar ninguna API. Para no saturar iNaturalist, un semáforo limita a 3 las llamadas concurrentes de esta API mientras las demás esperan su turno. Si una especie no existe en iNaturalist, también se guarda esa ausencia, evitando repetir la consulta inútil durante 30 días.
Motivación	Los datos de iNaturalist y OBIS son siempre los mismos independientemente de dónde se encuentre. Y un TTL de 30 días equilibra frescura (actualizaciones de las especies) con eficiencia.
Cuestiones abiertas	Si el estado IUCN de una especie cambia dentro del período de TTL de 30 días, la caché servirá el dato desactualizado. Aceptado dado que dichos cambios son poco frecuentes.
Alternativas	Caché solo en L1: resuelve repeticiones en la misma zona, pero si la misma especie aparece en el Mediterráneo y en el Atlántico Norte, se vuelven a hacer todas las llamadas. Pre-carga batch: mantener un catálogo de especies actualizado es inviable dado que las zonas de buceo son heterogéneas y las especies que aparecen varían enormemente.

Cuadro 7.7: Memorando técnico 004: Caché L2: SpeciesEnrichmentCache

Memorando técnico 005: Autenticación OAuth2 + JWT	
Asunto	La aplicación necesita identificar usuarios sin gestionar contraseñas ni sesiones de servidor.
Resumen	Validación del ID token de Google mediante su API de tokeninfo y emisión de JWT firmado con HMAC-SHA256, TTL 7 días.
Factores causantes	Scubex no tiene servidor de sesiones, así que la identidad del usuario tiene que viajar en cada petición. Implementar registro propio con contraseñas añadiría complejidad sin aportar nada al usuario.
Solución	El usuario hace click en “Continuar con Google”; Google devuelve un token que el backend valida directamente contra su API. Si es válido y el campo aud corresponde a la app (evitando suplantación), se crea o recupera el usuario y se emite un JWT propio firmado con HMAC-SHA256 que el cliente usará en todas sus peticiones autenticadas.
Motivación	Sin sesiones de servidor: compatible con despliegue stateless en Railway. Sin contraseñas: menor superficie de ataque. Google verifica la identidad: Scubex no almacena credenciales sensibles.
Cuestiones abiertas	Sin <i>refresh tokens</i> : tras 7 días el usuario debe volver a autenticarse. Sin revocación de tokens individuales (logout invalida solo la copia en localStorage).
Alternativas	Auth0 / Firebase Authentication : delega toda la lógica de autenticación a un servicio externo, simplificando la implementación, pero introduce dependencia y coste si el uso escala.

Cuadro 7.8: Memorando técnico 005: Autenticación OAuth2 + JWT

Flujo del escáner de biodiversidad

Con las cuatro decisiones en su lugar, el flujo completo al pulsar el botón de escaneo es el siguiente:

1. El servidor redondea las coordenadas del punto elegido y comprueba si existe un escaneo reciente de esa zona en base de datos. Si lo hay, devuelve el resultado directamente sin llamar a ninguna API.
2. Si no hay caché, construye el polígono WKT y consulta OBIS para obtener las especies registradas en esa zona.
3. Para cada especie, comprueba la caché de enriquecimiento: si ya existe, se recupera al instante.
4. Las especies nuevas se consultan en iNaturalist con un máximo de tres llamadas en paralelo para no saturar el servicio. Las que iNaturalist no reconoce se descartan y se guarda ese descarte durante 30 días para no volver a consultarlas.
5. Las especies reconocidas, con foto y nombre común, reciben además los datos ecológicos de OBIS (profundidad, temperatura, categoría de conservación) y se guardan en la caché de enriquecimiento.
6. El resultado completo del escaneo se persiste en base de datos para que la próxima consulta sobre esa zona sea inmediata.

Flujo de autenticación

El flujo de autenticación con Google es el siguiente:

1. El usuario hace clic en “Continuar con Google” y autoriza el acceso. Google devuelve un token de identidad al frontend.
2. El frontend envía ese token al backend. El servidor lo valida directamente contra la API de Google para confirmar que es auténtico y fue emitido para esta aplicación concretamente, no para otra.
3. Si la validación falla, se rechaza la petición con error de autenticación.
4. Si es correcta, el servidor busca al usuario en base de datos por su identificador de Google: si ya existe lo recupera; si es la primera vez, lo registra automáticamente.

5. Se emite un token propio firmado con validez de 7 días. El cliente lo guarda y lo incluye en todas sus peticiones posteriores como credencial.

7.4 DESPLIEGUE

Al cierre de esta iteración la aplicación quedó desplegada en la infraestructura descrita en el Arranque del proyecto:

- **Backend (Railway):** el proceso de arranque ejecuta `mvn package` y lanza el JAR resultante. Las variables de entorno `GOOGLE_CLIENT_ID`, `JWT_SECRET`, `DATABASE_URL` y `CORS_ALLOWED_ORIGINS` se configuran desde el panel de Railway.
- **Frontend (Vercel):** cada push a main desencadena un despliegue automático. La variable `VITE_API_BASE_URL` apunta a la URL pública del backend en Railway y `VITE_MAPTILER_KEY` provee acceso a las teselas marineras de MapTiler.
- **Base de datos (PostgreSQL en Railway):** accedida mediante `DATABASE_URL`; Hibernate gestiona el esquema automáticamente (`ddl-auto=update`).

La aplicación es accesible públicamente en `https://scubex.vercel.app` desde la finalización de esta iteración.

7.5 PRUEBAS

Tras realizar el código, se han creado tests unitarios que cubren los cinco problemas descritos en la sección de implementación y verifican los pasos clave de los flujos de escáner y autenticación.

Se han implementado más pruebas unitarias (validación de parámetros de entrada, casos límite de caché, controladores) que no se detallan aquí por no añadir ruido visual y técnico en la memoria.

Escáner de biodiversidad

- **Geometría WKT** (Tabla §7.4, paso 2 del flujo): dado el punto (36.5, -4.0) y radio 5000m, verifica que la URI generada contiene un POLYGON cerrado con al menos 4 vértices.

- **Filtrado de microorganismos** (Tabla §7.5, paso 4 del flujo): OBIS devuelve *Pycnococaceae*; iNaturalist responde `total_results=0`. Verifica que el resultado es una lista vacía.
- **Enriquecimiento con foto** (Tabla §7.5, paso 4 del flujo): OBIS devuelve *Chimaera monstrosa*; iNaturalist devuelve nombre común y foto. Verifica que `commonName` y `photoUrl` están presentes en la respuesta.
- **Datos ecológicos OBIS** (Tabla §7.7, paso 5 del flujo): *Octopus vulgaris* con entrada válida en iNaturalist. Verifica que profundidad, temperatura y categoría IUCN llegan correctamente en el `SpeciesResponse`.
- **Resiliencia ante fallo parcial de iNaturalist** (paso 4 del flujo): OBIS devuelve *Octopus vulgaris* y *Sepia officinalis*; la primera llamada a iNaturalist lanza `RestClientException` y la segunda funciona. Verifica que el resultado contiene exactamente una especie con todos sus campos.

Autenticación OAuth2 + JWT

- **Token sin credencial** (Tabla §7.8, paso 3 del flujo): petición sin campo `credential` devuelve 400 `Bad Request`.
- **Token rechazado por Google** (Tabla §7.8, paso 3 del flujo): Google no reconoce el token. Verifica que el servidor devuelve 401 `Unauthorized`.
- **Login válido completo** (Tabla §7.8, pasos 4–5 del flujo): token válido con `aud` correcto. Verifica que la respuesta es 200 `OK` con JWT y datos del usuario.
- **JWT: claims correctos y validación** (`JwtServiceTest`): verifica que el token generado contiene `googleId`, `email`, `name` y `picture`, y que un token manipulado lanza excepción.

ITERACIÓN 2: SISTEMA CLIMÁTICO

*E*sta iteración añade la segunda parte del MVP de Scubex: El panel de condiciones meteorológicas y marinas, el usuario puede consultar en tiempo real si una zona es apta o no para bucear antes de meterse al agua.

8.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Panel de clima con datos atmosféricos y marinos en tiempo real, evaluación automática de condiciones de buceo y caché de 30 minutos. Ver Tabla §8.1.

Con el mapa ya operativo desde la iteración anterior, la siguiente pieza del MVP es la información climática: antes de cualquier inmersión, el buceador necesita saber si el mar está en calma. El panel proporciona esa respuesta en un único vistazo, evaluando automáticamente las condiciones sin que el usuario tenga que interpretar valores crudos ni consultar varias fuentes.

Análisis de valor aportado: Panel de Clima	
Propuesta	Consulta del estado del mar y la atmósfera para el punto marcado en el mapa, con evaluación automática de si las condiciones son aptas para bucear.
Valor	Un buceador no puede planificar una inmersión sin saber si el mar está en calma. El panel proporciona en un único vistazo los datos que determinan la viabilidad de la salida olas, corriente, visibilidad y tiempo sin que el usuario tenga que consultar varias fuentes ni interpretar valores crudos.
Coste	Integración con dos endpoints de Open-Meteo (atmospheric forecast y marine API), ambos gratuitos y sin clave. El único esfuerzo no trivial es definir los criterios de evaluación: qué umbrales y pesos determinan si una condición es buena, tolerable o adversa desde el punto de vista del buceo.
Opciones	Windy: mapas muy visuales de viento y oleaje, pero no tiene API pública gratuita para datos en crudo. Evaluación manual: mostrar solo los valores numéricos y dejar que el usuario decida; eliminaba la lógica del servidor pero obligaba al usuario a conocer los umbrales de seguridad.
Riesgos	Open-Meteo no ofrece SLA; una caída devuelve error sin datos de clima. Los datos marinos no existen en zonas continentales o bahías cerradas, por lo que esos campos pueden llegar vacíos.
Deuda técnica	Solo se muestran condiciones en el instante actual; no hay previsión por horas ni por días. Los umbrales de evaluación son globales e iguales para todos los usuarios, sin distinción entre perfiles (principiante vs. avanzado).

Cuadro 8.1: Análisis de valor aportado: Panel de Clima

8.2 DISEÑO

El diagrama UML de la Fig. §8.1 muestra las relaciones entre los componentes del sistema climático: controlador, servicio, entidad de caché y los dos clientes de Open-Meteo.

El diseño separa claramente la obtención de datos meteorológicos de la lógica de evaluación de condiciones, lo que permite modificar los criterios de aptitud para el buceo sin tocar la capa de integración con Open-Meteo.

8.3 IMPLEMENTACIÓN

El sistema climático consulta dos APIs de Open-Meteo para combinar datos atmosféricos y marinos, aplica una puntuación ponderada para determinar si las condiciones son aptas para bucear, y guarda el resultado en caché para evitar llamadas repetidas. Los tres desafíos técnicos y las decisiones adoptadas para resolverlos están recogidos en las tablas siguientes:

- **Dos fuentes de datos** (Tabla §8.2): los datos atmosféricos y los marinos provienen de dos endpoints distintos de Open-Meteo; se fusionan en un único objeto de respuesta con tolerancia a que la API marina no tenga datos para la zona.
- **Evaluación automática de condiciones** (Tabla §8.3): el usuario necesita saber si puede bucear, no interpretar valores crudos. Se aplica una puntuación ponderada con anulaciones críticas ante los factores más peligrosos.
- **Caché de 30 minutos** (Tabla §8.4): el mapa es interactivo y el usuario puede mover el punto libremente; sin caché, cada pequeño desplazamiento generaría dos peticiones a Open-Meteo.

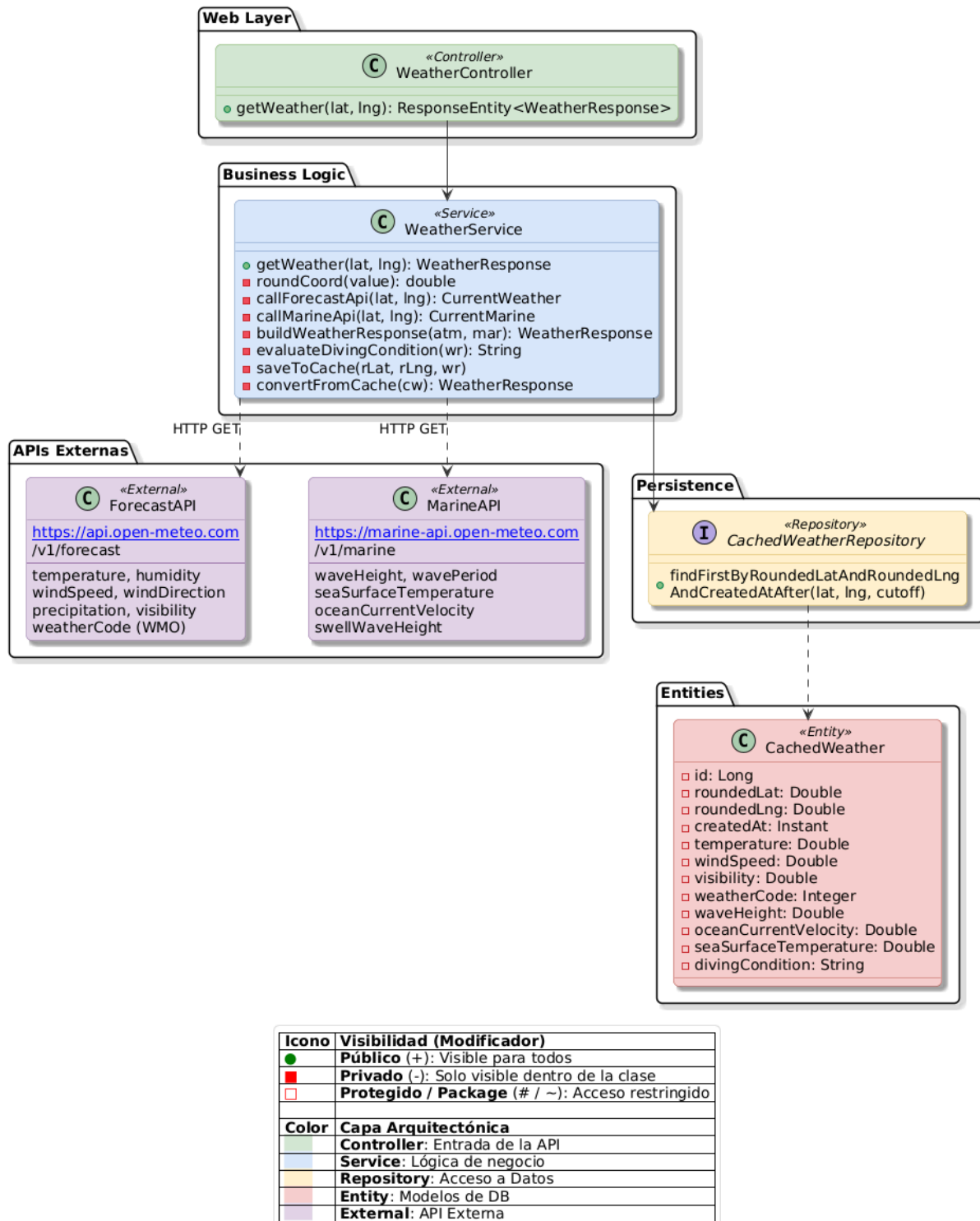


Figura 8.1: Diagrama UML de diseño para la iteración 2

Memorando técnico 006: WeatherResponse — Datos de Dos Fuentes	
Asunto	Los datos atmosféricos (temperatura, viento, lluvia) y los datos marinos (olas, corrientes, temperatura del agua) provienen de dos endpoints distintos de Open-Meteo.
Resumen	Se realizan dos llamadas independientes y sus resultados se fusionan en un único objeto WeatherResponse que el frontend consume en una sola petición.
Factores causantes	Open-Meteo separa su API en dos servicios: el de previsión atmosférica (<code>api.open-meteo.com</code>) y el marino (<code>marine-api.open-meteo.com</code>). El panel de Scubex necesita datos de ambos para ser útil.
Solución	WeatherService llama de forma independiente a <code>callForecastApi()</code> y <code>callMarineApi()</code> , y combina los resultados con <code>buildWeatherResponse()</code> . Si una de las dos APIs falla o devuelve nulo (por ejemplo, la marina en zonas continentales), los campos correspondientes quedan vacíos en la respuesta pero el resto de datos se sigue devolviendo correctamente.
Motivación	Principalmente la riqueza de los datos, tener datos atmosféricos y marinos es la única manera que la implementación de datos climáticos pueda ser realmente útil. Además separar las dos llamadas permite tolerancia a fallos parciales: si la API marina no tiene datos para una zona interior, el panel sigue mostrando temperatura, viento y lluvia, que sí están disponibles en cualquier punto del planeta.
Cuestiones abiertas	Las dos llamadas se realizan de forma secuencial. Paralelizarlas con <code>CompletableFuture</code> reduciría la latencia en los casos sin caché.
Alternativas	Windy Meteo API: requiere suscripción de pago para acceso a datos en crudo.

Cuadro 8.2: Memorando técnico 006: WeatherResponse

Memorando técnico 007: Evaluación Automática de Condiciones de Buceo	
Asunto	El usuario necesita saber si puede bucear sin tener que interpretar él mismo una tabla de valores meteorológicos.
Resumen	Un algoritmo de puntuación ponderada clasifica las condiciones en <i>buenas</i> , <i>tolerables</i> o <i>adversas</i> , con anulaciones directas ante valores extremos en los factores más críticos.
Factores causantes	Un buceador no pide saber exactamente que hay olas de 1,3 m; quiere saber si puede entrar al agua. Presentar solo los valores crudos obliga al usuario a conocer umbrales de seguridad que varían según su experiencia y el tipo de buceo.
Solución	<code>evaluateDivingCondition()</code> aplica primero un conjunto de anulaciones críticas: oleaje > 2,5 m, corriente > 5 km/h, viento > 40 km/h, tormenta eléctrica (código WMO 95/96/99) o visibilidad < 1 000 m devuelven inmediatamente "bad". Si no se cumple ninguna anulación, se puntúan seis factores con pesos distintos (oleaje = 3, corriente = 3, visibilidad = 2, viento = 2, código meteorológico = 2, probabilidad de lluvia = 1) y se calcula la media ponderada: $\geq 1,5$ es <i>buenas</i> , $\geq 0,8$ es <i>tolerables</i> , resto <i>adversas</i> .
Motivación	El oleaje y la corriente son los factores más peligrosos para un buceador; por eso tienen el peso más alto y los umbrales de anulación más conservadores. La probabilidad de lluvia tiene el peso mínimo porque no afecta directamente a la seguridad bajo el agua.
Cuestiones abiertas	Los umbrales son fijos e iguales para todos los usuarios. Un perfil de "buceador avanzado" podría tolerar corrientes de hasta 4 km/h sin problema, mientras que un principiante debería ver <i>adversas</i> con corriente de 2 km/h.
Alternativas	Umbrales fijos sin ponderación: más simple, pero no tiene una correspondencia correcta con los factores del buceo, el oleaje siempre importará mas que si va a llover.

Cuadro 8.3: Memorando técnico 007: Evaluación de Condiciones

Memorando técnico 008: Caché de Clima (CachedWeather)

Asunto	Cada vez que el usuario mueve el mapa o cambia el punto, se lanzarían dos llamadas a Open-Meteo, añadiendo latencia innecesaria.
Resumen	Los datos de clima se guardan en base de datos durante 30 minutos por coordenadas redondeadas, evitando repetir las llamadas para puntos cercanos o consultas frecuentes sobre la misma zona.
Factores causantes	El mapa es interactivo y el usuario puede mover el punto libremente. Sin caché, cada pequeño desplazamiento generaría dos peticiones HTTP hacia Open-Meteo, con la latencia acumulada correspondiente.
Solución	Se aplica el mismo esquema de redondeo de coordenadas descrito en el Memorando §7.6: las coordenadas se redondean antes de usarlas como clave de búsqueda, agrupando puntos cercanos bajo la misma entrada. Si existe un registro de menos de 30 minutos para esas coordenadas en la tabla <code>cached_weather</code> , se devuelve directamente sin tocar Open-Meteo. El TTL de 30 minutos equilibra frescura de los datos climáticos con eficiencia: el tiempo meteorológico no cambia de forma significativa en ventanas menores.
Motivación	Los datos de clima son mucho más volátiles que los de biodiversidad, por lo que el TTL es 30 minutos frente a las 48 horas del caché de escaneos (Memorando §7.6). Reutilizar el mismo mecanismo de caché en base de datos evita añadir nuevos servicios: la tabla <code>cached_weather</code> sigue exactamente el mismo patrón que <code>CachedScan</code> .
Cuestiones abiertas	El caso límite del redondeo (dos puntos muy próximos que caen en celdas distintas) es el mismo ya reconocido en el Memorando §7.6 para el caché de escaneos, y se acepta por la misma razón: alta complejidad de resolución, para un bajo impacto.
Alternativas	TTL más corto (5–10 min): más fresco, pero aumenta significativamente las llamadas a Open-Meteo. Sin caché: máxima frescura, pero penaliza la experiencia en sesiones de exploración del mapa.

Flujo del sistema climático

Al hacer clic sobre el mapa y activar el panel de clima, el flujo completo es el siguiente:

1. El servidor redondea las coordenadas a dos decimales y comprueba si existe un resultado reciente (menos de 30 minutos) para esa zona en base de datos. Si lo hay, lo devuelve directamente sin llamar a ninguna API.
2. Si no hay caché, consulta la API atmosférica de Open-Meteo: temperatura, humedad, viento, precipitación, visibilidad y código meteorológico WMO.
3. Consulta la API marina: altura de ola, corriente oceánica, temperatura del agua y oleaje de fondo. En zonas continentales esta llamada devuelve vacío y esos campos quedan sin datos, pero el resto de la respuesta sigue siendo válida.
4. Los datos de ambas fuentes se fusionan en una única respuesta.
5. Se evalúan las condiciones de buceo: si algún factor supera un umbral crítico (oleaje $> 2,5$ m, corriente > 5 km/h, viento > 40 km/h, tormenta eléctrica o visibilidad $< 1\,000$ m), el resultado es inmediatamente adverso sin calcular la puntuación. En caso contrario, seis factores con pesos distintos determinan si las condiciones son buenas, tolerables o adversas.
6. El resultado completo —con los datos de ambas APIs y la evaluación de condiciones— se guarda en base de datos bajo las coordenadas redondeadas, con un tiempo de vida de 30 minutos.
7. La respuesta se devuelve al cliente. Las próximas consultas sobre esa misma zona durante los siguientes 30 minutos se resolverán en el paso 1 sin llamar a ninguna API.

8.4 DESPLIEGUE

Esta iteración no requirió cambios en la infraestructura existente. Open-Meteo es una API pública sin autenticación, por lo que no fue necesario añadir nuevas variables de entorno ni credenciales. La tabla `cached_weather` fue creada automáticamente por Hibernate al arrancar el servicio en Railway.

Las URLs de los dos endpoints de Open-Meteo se externalizan en `application.properties` mediante las propiedades `open-meteo.weather.url` y `open-meteo.marine.url`, lo que permite cambiarlas sin recompilar.

8.5 PRUEBAS

Las pruebas cubren los tres desafíos técnicos documentados y verifican los pasos clave del flujo del sistema climático.

Sistema climático

- **Caché activa** (Tabla §8.4, paso 1 del flujo): se inyecta una entrada de 10 minutos de antigüedad. Verifica que el servicio devuelve los datos cacheados y no realiza ninguna llamada a Open-Meteo.
- **Anulación crítica por oleaje** (Tabla §8.3, paso 5 del flujo): la API marina devuelve oleaje de 3,0 m con condiciones atmosféricas ideales. Verifica que las condiciones de buceo son adversas, confirmando que la anulación crítica se dispara antes de calcular la puntuación ponderada.
- **Zona continental sin datos marinos** (Tabla §8.2, pasos 3–4 del flujo): la API marina devuelve cuerpo nulo (coordenadas de Madrid). Verifica que no se lanza excepción, que los campos marinos son nulos y que las condiciones se evalúan correctamente solo con datos atmosféricos.
- **Tolerancia a fallo de la API atmosférica** (Tabla §8.2, paso 2 del flujo): la llamada a Open-Meteo atmosférico lanza `RestClientException`. Verifica que el servicio no propaga el error, que los campos atmosféricos son nulos y que los datos marinos siguen presentes.

De igual forma que la anterior iteración proyecto cuenta con más pruebas unitarias (controlador de clima, validación de parámetros de entrada) que no se detallan aquí por no estar directamente vinculadas a las decisiones documentadas en esta sección.

ITERACIÓN 3: PUBLICACIONES

***E**sta iteración añade el final del MVP de Scubex: Un sistema de publicaciones. Los usuarios autenticados pueden crear publicaciones geolocalizadas con imagen desde cualquier punto del mapa, y cualquier visitante puede consultarlas directamente desde los marcadores del mapa.*

9.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Publicaciones geolocalizadas: crear, editar y eliminar entradas con título, descripción e imagen, ancladas a un punto del mapa. Ver Tabla §9.1.

Con el mapa operativo y la autenticación en su lugar, la última pieza del MVP es el contenido generado por los propios usuarios. Sin publicaciones, Scubex es una herramienta de consulta; con ellas, se convierte en un diario comunitario de inmersiones donde los usuarios pueden ver que han encontrado otros buceadores en un punto específico del mapa.

Análisis de valor aportado: Publicaciones Geolocalizadas	
Propuesta	Un sistema de publicaciones integrado en el mapa: el usuario hace clic en cualquier punto, escribe un título, una descripción y adjunta una foto, y la publicación queda anclada a esas coordenadas. Cualquier persona puede verlas explorando el mapa.
Valor	Convierte Scubex en un diario colectivo de inmersiones. Un buceador puede compartir qué encontró en una cala concreta, y otros usuarios descubren ese contenido simplemente explorando la zona en el mapa, sin necesidad de buscar por texto ni conocer el nombre del lugar.
Coste	Diseñar el modelo de datos con coordenadas indexadas, gestionar el almacenamiento de imágenes en base de datos, implementar el filtrado por área visible y resolver el nombre del lugar a partir de las coordenadas en el frontend.
Opciones	Feed cronológico sin mapa: más simple de implementar, pero pierde la dimensión geográfica que diferencia Scubex de cualquier otra red social. Spots predefinidos: el usuario solo puede publicar en zonas conocidas de antemano, lo que elimina la libertad de compartir descubrimientos propios.
Riesgos	Si el usuario hace zoom muy hacia fuera, el área visible es tan grande que la respuesta podría incluir un volumen muy alto de publicaciones. Tampoco hay ningún control sobre dónde se publica: un usuario puede anclar una publicación en tierra firme o en cualquier punto sin relación con el buceo.
Deuda técnica	Sin límite de área en la consulta. Sin moderación de contenido ni restricción geográfica.

Cuadro 9.1: Análisis de valor aportado: Publicaciones Geolocalizadas

9.2 DISEÑO

El diagrama UML de la Fig. §9.1 muestra los componentes del sistema de publicaciones: controladores, servicio, repositorios y entidades, así como la dependencia del módulo de geocoding en el frontend.

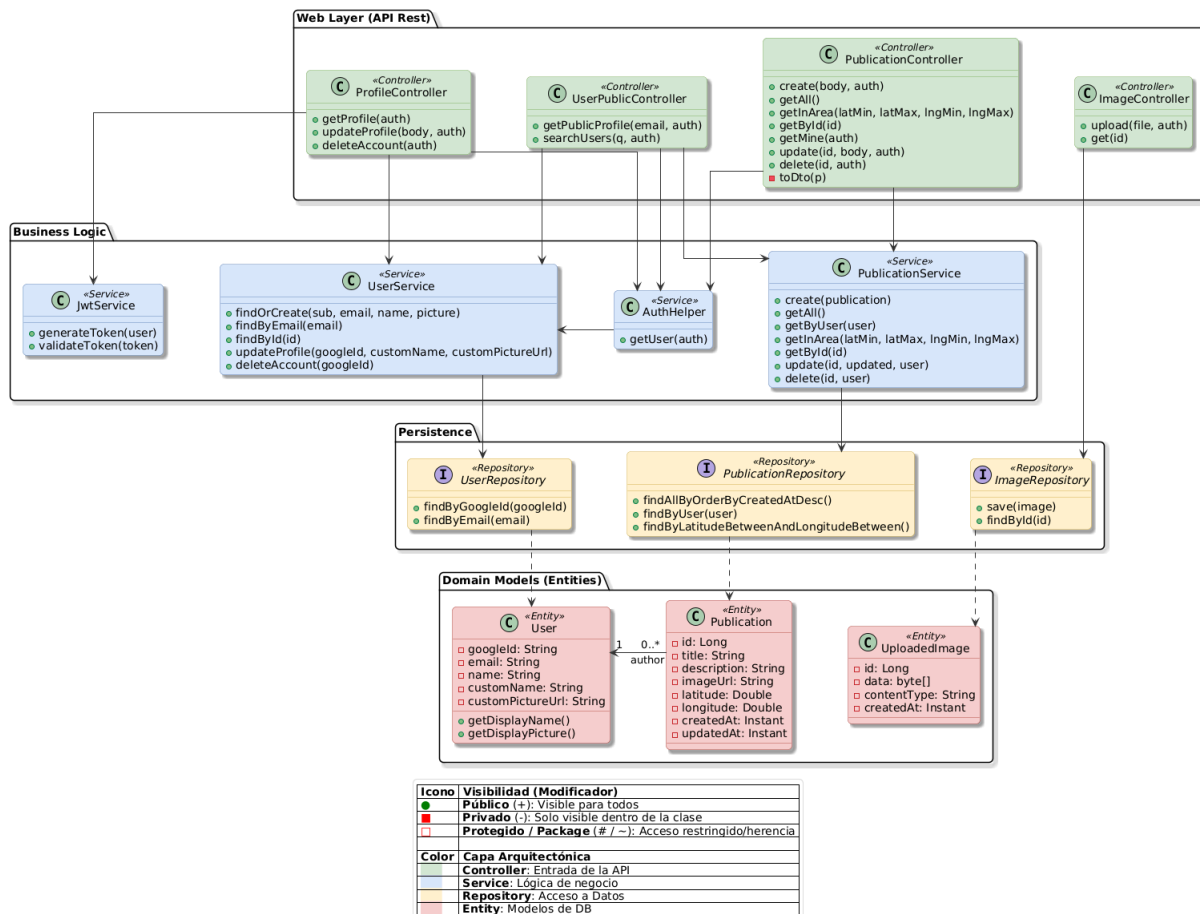


Figura 9.1: Diagrama UML de diseño para la iteración 3

El diagrama muestra la separación entre el controlador, que valida la autoría antes de delegar en el servicio, y el servicio, que centraliza el filtrado por coordenadas y la gestión de imagen. La lógica de geocoding inverso reside íntegramente en el frontend: convertir coordenadas a nombre legible no requiere persistencia en el servidor y no tiene sentido moverla al backend.

9.3 IMPLEMENTACIÓN

El sistema de publicaciones necesita escalar con el volumen de contenido sin degradar el rendimiento, ofrecer contexto geográfico legible al usuario y almacenar imágenes de forma persistente sin servicios adicionales. Los tres desafíos técnicos y las decisiones adoptadas para resolverlos están recogidos en las tablas siguientes:

- **Filtrado por área visible** (Tabla §9.2): descargar todas las publicaciones en cada consulta del mapa haría la aplicación más lenta conforme crece el contenido. El servidor devuelve solo las publicaciones dentro del área visible, apoyado en un índice de coordenadas.
- **Geocoding inverso** (Tabla §9.3): las coordenadas crudas no aportan contexto al usuario. El frontend traduce cada punto a un nombre de lugar mediante Nominatim y guarda el resultado en memoria de sesión para no repetir la consulta.
- **Almacenamiento de imágenes como BLOB** (Tabla §9.4): los contenedores de Railway no tienen disco persistente. Las imágenes se guardan directamente en PostgreSQL y se sirven con caché de un año en el navegador.

Memorando técnico 009: Filtrado de Publicaciones por Viewport del Mapa	
Asunto	Descargar todas las publicaciones existentes en cada consulta del mapa haría la aplicación cada vez más lenta conforme crece el contenido.
Resumen	El servidor devuelve únicamente las publicaciones que caen dentro del área visible del mapa en cada momento, apoyándose en un índice de coordenadas para que la consulta sea eficiente.
Factores causantes	Si cada vez que se abre el mapa o se mueve se descargaran todas las publicaciones de la base de datos, la respuesta crecería indefinidamente con el tiempo. En zonas con mucho contenido, esto haría la experiencia lenta incluso para usuarios con buena conexión.
Solución	El frontend envía al servidor los límites del área visible (latitud y longitud mínima y máxima) con cada consulta. El servidor filtra y devuelve únicamente las publicaciones que caen dentro de ese rectángulo. Para que esta búsqueda sea eficiente sin importar cuántas publicaciones haya, las coordenadas de cada publicación tienen un índice en base de datos, de modo que la consulta no tiene que recorrer la tabla entera.
Motivación	Es la forma natural de escalar un sistema con contenido georreferenciado: el usuario no necesita lo que no puede ver en pantalla. El índice en coordenadas garantiza que añadir publicaciones no penalice el rendimiento de la consulta.
Cuestiones abiertas	Si el usuario hace zoom muy hacia fuera, los límites del mapa abarcan un área enorme y la respuesta podría incluir un número muy alto de publicaciones de golpe.
Alternativas	Agrupación geográfica: reduce la carga visual de marcadores en el mapa, pero no reduce la cantidad de datos transferidos.

Cuadro 9.2: Memorando técnico 009: Filtrado por Viewport del Mapa

Memorando técnico 010: Geocoding Inverso con Nominatim	
Asunto	Las publicaciones se anclan a coordenadas, pero mostrar latitud y longitud crudas no aporta contexto al usuario.
Resumen	El frontend traduce las coordenadas de cada publicación a un nombre de lugar legible consultando Nominatim, y guarda los resultados en memoria para no repetir la consulta durante la misma sesión.
Factores causantes	Tanto el formulario de creación como la vista de detalle muestran dónde está el punto marcado. Para un usuario es mucho más informativo leer “Punta de la Mona, Granada” que ver un par de números decimales.
Solución	Cuando se necesita el nombre de un punto, se consulta Nominatim probando primero el nivel de detalle más alto (playas, puntos de interés), luego el intermedio (ciudad o municipio) y finalmente el regional. Si ningún nivel devuelve resultado, se muestra “Mar abierto” como alternativa. Para no repetir la misma consulta si el usuario vuelve al mismo punto, los resultados se guardan en memoria durante toda la sesión.
Motivación	Nominatim es gratuito y no requiere ninguna clave de API, lo que lo convierte en la opción más directa. Probar niveles de detalle de mayor a menor garantiza el nombre más concreto posible sin lanzar varias peticiones en paralelo. Guardar los resultados en memoria es suficiente: si el usuario cierra la pestaña, la caché se limpia sola y en la siguiente sesión los datos de Nominatim seguirán siendo los mismos.
Cuestiones abiertas	Nominatim limita las peticiones a una por segundo para uso no comercial. En sesiones de exploración rápida del mapa, las peticiones de puntos que el usuario abandona antes de que se resuelvan se cancelan automáticamente, lo que ayuda a no saturar el servicio; pero sin una cola controlada podría alcanzarse el límite en casos extremos.
Alternativas	Google Maps Geocoding API: mayor precisión, pero de pago.

Cuadro 9.3: Memorando técnico 010: Geocoding Inverso con Nominatim

Memorando técnico 011: Almacenamiento de Imágenes como BLOB	
Asunto	Las publicaciones incluyen una fotografía adjunta que debe conservarse de forma permanente.
Resumen	Las imágenes se guardan como BLOB directamente en PostgreSQL. El servidor las sirve al navegador con una cabecera de caché de un año, de modo que una vez descargadas no se vuelven a pedir.
Factores causantes	Los contenedores de Railway no tienen almacenamiento en disco persistente: cualquier fichero guardado localmente desaparece al reiniciar el servidor. Guardar las imágenes en la propia base de datos resuelve el problema sin añadir ningún servicio externo.
Solución	Al subir una fotografía, el servidor comprueba que el formato sea aceptado (JPEG, PNG, GIF o WebP) y que no supere los 5 MB. Si pasa esa validación, los datos del fichero se guardan en la base de datos junto al tipo de formato. Para recuperarla, el servidor la lee de la base de datos y la envía al navegador indicándole que la almacene en caché durante un año, ya que una imagen identificada por su identificador nunca cambia.
Motivación	PostgreSQL ya forma parte del proyecto, así que no se necesita ningún servicio adicional ni credenciales extra. Para el volumen de imágenes esperado en Scubex la solución es perfectamente suficiente y evita introducir dependencias externas que habría que mantener.
Cuestiones abiertas	Las imágenes se sirven siempre al tamaño original, sin reducción ni compresión. No existe un límite de imágenes por usuario.
Alternativas	Amazon S3: más escalable a largo plazo, pero introduce coste y complejidad innecesarios para el alcance actual.

Cuadro 9.4: Memorando técnico 011: Almacenamiento de Imágenes como BLOB

Flujo de creación de una publicación

Cuando un usuario autenticado hace clic en el mapa y envía el formulario de nueva publicación, el flujo es el siguiente:

1. El usuario selecciona el punto en el mapa. El frontend resuelve automáticamente las coordenadas a un nombre de lugar legible mediante Nominatim y lo muestra en el formulario. Si el nombre ya fue consultado en esta sesión, se recupera de la memoria local sin llamar a Nominatim.
2. Si la publicación incluye una imagen, el frontend la envía primero al servidor. El servidor comprueba que el formato sea aceptado (JPEG, PNG, GIF o WebP) y que no supere los 5 MB, guarda los bytes en PostgreSQL y devuelve la URL que identifica la imagen.
3. El usuario confirma el formulario con título, descripción y, si se subió, la URL de la imagen. El servidor verifica que la sesión sigue activa y que los campos obligatorios están presentes.
4. La publicación se persiste con las coordenadas del punto, el autor y la fecha de creación.
5. La publicación aparece inmediatamente como marcador en el mapa para cualquier usuario que explore esa zona.

Flujo de consulta de publicaciones

Cuando el mapa se mueve o se carga por primera vez, las publicaciones visibles se obtienen así:

1. El frontend envía al servidor los límites del área visible del mapa (latitud y longitud mínima y máxima).
2. El servidor consulta la base de datos filtrando por esas coordenadas usando el índice. Solo se devuelven las publicaciones que caen dentro del rectángulo visible.
3. Cada publicación se devuelve con los datos del autor, las coordenadas y la URL de imagen si la tiene.

4. El frontend coloca un marcador en el mapa por cada publicación. Al hacer clic en uno, se carga el detalle completo de la publicación.
5. Si la publicación tiene imagen, el navegador la solicita por su URL. El servidor la sirve con una cabecera de caché de un año: después de la primera descarga, el navegador no vuelve a pedirla.

9.4 DESPLIEGUE

Esta iteración añadió dos nuevas tablas a la base de datos: `publications` y `uploaded_images`, ambas creadas automáticamente por Hibernate al arrancar el servicio en Railway. No fue necesario añadir nuevas variables de entorno ni credenciales, ya que el almacenamiento de imágenes se realiza en la misma base de datos PostgreSQL existente.

9.5 PRUEBAS

Las pruebas cubren los tres desafíos técnicos documentados y verifican los pasos clave de los dos flujos.

Publicaciones

- **Filtrado por área** (Tabla §9.2, paso 2 del flujo de consulta): el repositorio recibe un bounding box concreto y devuelve dos publicaciones. Verifica que el servicio retorna exactamente esas dos y que la consulta se ejecuta con los límites correctos.
- **Edición por propietario ajeno** (Tabla §9.2, paso 3 del flujo de creación): un usuario distinto al autor intenta editar la publicación. Verifica que el resultado es nulo y que no se persiste ningún cambio.
- **Eliminación por propietario ajeno** (Tabla §9.2): un usuario distinto al autor intenta borrar la publicación. Verifica que el resultado es falso y que no se ejecuta el borrado.

Imágenes

- **Subida sin autenticación** (Tabla §9.4, paso 2 del flujo de creación): sin sesión activa, devuelve 401 sin persistir nada.

- **Validaciones de archivo** (Tabla §9.4, paso 2 del flujo de creación): cuatro casos independientes verifican que un archivo vacío, uno superior a 5 MB, uno sin tipo declarado y uno con formato no permitido (PDF) devuelven 400 sin persistir nada.
- **Subida válida** (Tabla §9.4, paso 2 del flujo de creación): un JPEG válido con usuario autenticado devuelve 200 con la URL de la imagen y persiste exactamente una imagen en base de datos.
- **Descarga de imagen existente** (Tabla §9.4, paso 5 del flujo de consulta): una imagen existente devuelve 200, el Content-Type correcto y los bytes originales.
- **Descarga de imagen inexistente** (Tabla §9.4, paso 5 del flujo de consulta): un identificador que no existe en base de datos devuelve 404.

El proyecto cuenta con más pruebas unitarias (controlador de publicaciones, listado, edición, control de autorización) que no se detallan aquí para no añadir exceso de información. Al no estar directamente vinculadas a los desafíos técnicos documentados en esta sección.

ITERACIÓN 4: INTERACCIONES Y RED SOCIAL

*E*sta iteración convierte Scubex en una red social de buceo. Se añaden likes, guardados y comentarios con menciones en las publicaciones, la posibilidad de seguir a otros buceadores con perfiles públicos, y un centro de notificaciones que informa al usuario de toda actividad relacionada con su contenido.

10.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Interacciones con publicaciones: likes, guardados y comentarios con soporte para menciones a otros usuarios. Ver Tabla §10.1.
2. Seguimiento de usuarios y perfiles públicos: seguir a otros buceadores y consultar su perfil con contadores de seguidores y seguidos. Ver Tabla §10.2.
3. Centro de notificaciones: campana con contador de no leídas que agrupa likes, comentarios, menciones y nuevos seguidores. Ver Tabla §10.3.

Con las publicaciones en el mapa, el siguiente paso natural es que los usuarios puedan reaccionar al contenido de los demás. Las interacciones dan retroalimentación al autor; los follows construyen la red entre buceadores; y las notificaciones cierran el ciclo informando al usuario de todo lo que ocurre con su contenido, sin que tenga que ir a buscarlo.

Análisis de valor aportado: Interacciones con Publicaciones	
Propuesta	Añadir botones de like y guardado a cada publicación del mapa, y una sección de comentarios con soporte para mencionar a otros usuarios por nombre.
Valor	Da retroalimentación directa al autor y ayuda a otros usuarios a identificar el contenido más interesante. El guardado permite construir una lista personal de inmersiones que el usuario quiere visitar. Los comentarios abren la posibilidad de preguntar detalles, compartir experiencias propias en ese mismo punto o mencionar a alguien que puede estar interesado.
Coste	Tres tablas nuevas en base de datos (likes, guardados, comentarios), lógica de toggle para que el mismo botón active y desactive la acción sin duplicar registros, y análisis del texto de los comentarios para resolver las menciones y notificar a los usuarios citados.
Opciones	Solo likes sin comentarios: más simple de implementar, pero elimina la conversación entre buceadores que enriquece el contenido. Reacciones con emojis: más expresivas, pero añaden complejidad de diseño e implementación sin un beneficio claro para la comunidad de buceo.
Riesgos	Sin moderación ni filtrado de contenido, cualquier texto puede publicarse como comentario. No hay control sobre el volumen de comentarios por publicación.
Deuda técnica	Sin paginación de comentarios. Sin posibilidad de editar un comentario ya publicado. Sin sistema de denuncias de contenido inapropiado.

Cuadro 10.1: Análisis de valor aportado: Interacciones con Publicaciones

Análisis de valor aportado: Seguimiento de Usuarios y Perfiles Públicos	
Propuesta	Permitir seguir a otros usuarios y consultar su perfil público con nombre, foto y contadores de seguidores y seguidos.
Valor	Construye la dimensión social de Scubex. Un buceador puede seguir a quien publica habitualmente en sus zonas favoritas y, con el tiempo, formar parte de una comunidad sin necesidad de conocerse fuera de la aplicación. El perfil público da identidad a cada autor: el nombre y la foto dan contexto a las publicaciones del mapa.
Coste	Tabla de follows con comprobación de que un usuario no puede seguirse a sí mismo, endpoint de perfil público accesible, componentes de listas de seguidores y seguidos, y actualización optimista en la interfaz para que el botón responda de forma inmediata sin esperar al servidor.
Opciones	Amistad bidireccional: ambas partes deben aceptarse mutuamente, lo que es más privado pero pierde la asimetría característica de las redes de interés. Sin sistema de follows: el contenido sería completamente anónimo y no habría forma de seguir a un autor concreto ni de construir comunidad.
Riesgos	Los perfiles son completamente públicos: nombre, foto y publicaciones son visibles para cualquier visitante sin necesidad de cuenta. No hay opción de perfil privado.
Deuda técnica	Sin perfiles privados ni opción de aprobar seguidores. Sin bloqueos entre usuarios.

Cuadro 10.2: Análisis de valor aportado: Seguimiento de Usuarios y Perfiles Públicos

Análisis de valor aportado: Centro de Notificaciones	
Propuesta	Campana en la barra de navegación con contador de no leídas que abre un panel con las notificaciones de likes, comentarios, menciones y nuevos seguidores; cada una puede eliminarse deslizándola.
Valor	El usuario sabe si alguien interactuó con su contenido sin tener que revisar manualmente cada publicación. El contador visible en la campana mantiene el engagement con la aplicación sin requerir recarga de página.
Coste	Tabla de notificaciones con cuatro tipos (like, comentario, mención, follow) y consulta periódica del contador desde el frontend.
Opciones	Sin notificaciones: el usuario solo descubre las interacciones si revisa manualmente sus propias publicaciones, lo que reduce significativamente el engagement.
Riesgos	El intervalo de polling introduce un pequeño retraso entre el evento y la notificación visible. Sin agrupación, una publicación popular puede generar un gran número de entradas individuales en el panel.
Deuda técnica	Sin notificaciones push al navegador cuando la pestaña está en segundo plano. Notificaciones del mismo tipo no se agrupan: diez likes de distintos usuarios generan diez entradas separadas. No existe opción de silenciar notificaciones por tipo ni por publicación concreta.

Cuadro 10.3: Análisis de valor aportado: Centro de Notificaciones

10.2 DISEÑO

El diagrama UML de la Fig. §10.1 muestra los componentes de la capa social: controladores de interacciones y notificaciones, servicios, repositorios y entidades, así como las dependencias entre *InteractionService* y *NotificationService*.

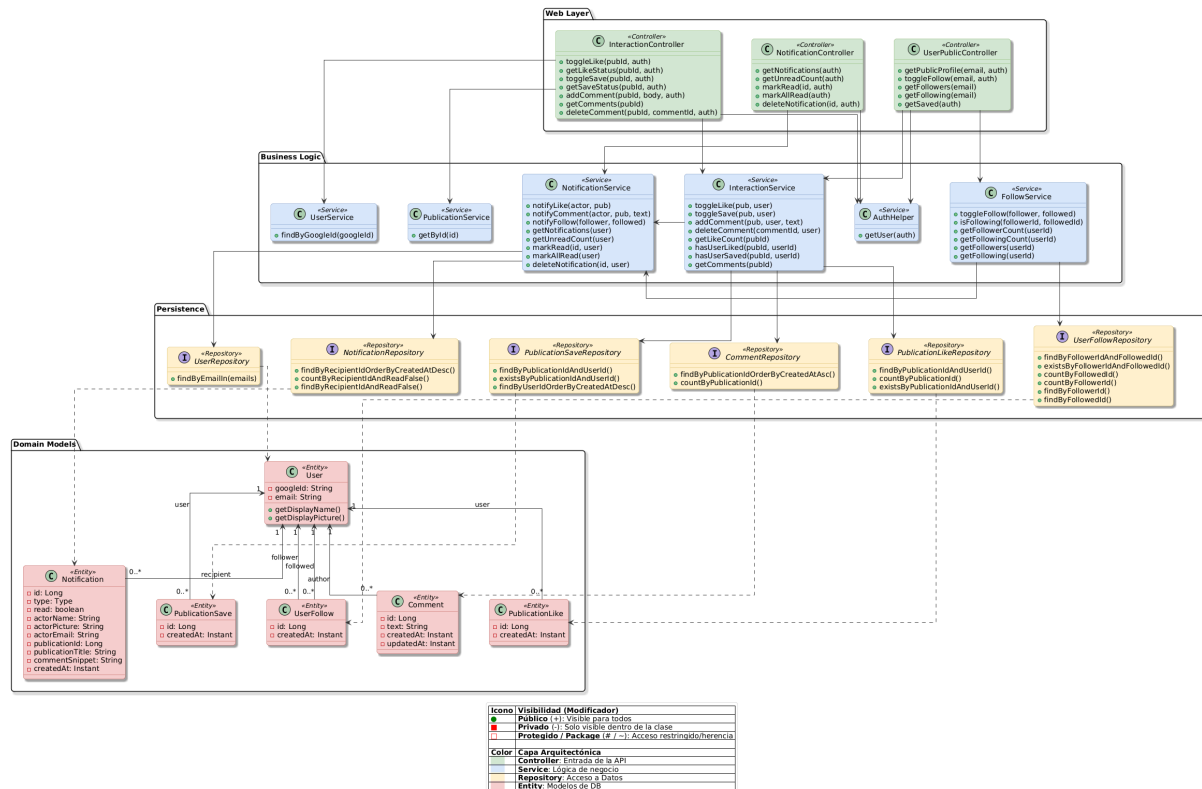


Figura 10.1: Diagrama UML de diseño para la iteración 4

El diseño concentra toda la lógica social en dos servicios independientes: uno gestiona las acciones directas sobre publicaciones (likes, guardados, comentarios) y el otro centraliza la generación de notificaciones. Esta separación permite evolucionar la lógica de notificaciones sin tocar la capa de interacciones.

10.3 IMPLEMENTACIÓN

La capa social introduce tres desafíos que no existían en iteraciones anteriores: gestionar acciones reversibles sin duplicar datos, enviar notificaciones precisas sin repetirlas, y mantener el contador de notificaciones actualizado sin conexiones permanentes

al servidor. Los tres desafíos técnicos y las decisiones adoptadas para resolverlos están recogidos en las tablas siguientes:

- **Interacciones con toggle** (Tabla §10.4): dar like varias veces crearía registros duplicados y notificaciones repetidas. El servidor comprueba si ya existe el registro antes de actuar: si existe lo borra, si no lo crea.
- **Menciones y deduplicación de notificaciones** (Tabla §10.5): los nombres de usuario no son únicos, y el propietario de una publicación podría recibir dos notificaciones del mismo comentario si también aparece mencionado. Las menciones se codifican con el email y un conjunto de ya-notificados evita los duplicados.
- **Centro de notificaciones con polling** (Tabla §10.6): el cliente no sabe cuándo llega una notificación nueva. La campana consulta periódicamente al servidor el número de no leídas y actualiza el contador sin necesidad de conexión permanente.

Memorando técnico 012: Implementación de Likes, Guardados y Comentarios

Asunto	Las publicaciones admiten tres tipos de interacción: like, guardado y comentario, cada uno con su propio botón. Likes y guardados pueden deshacerse; los comentarios se añaden siempre como entrada nueva.
Resumen	Para likes y guardados, el servidor mira si ya existe el registro antes de actuar: si ya está, lo borra; si no, lo crea. Así nunca se acumulan registros incorrectos. Los comentarios siempre generan una entrada nueva y solo el propio autor puede borrarlos.
Factores causantes	Sin la comprobación previa, dar like varias veces crearía varios registros en base de datos y dispararía una notificación al autor por cada uno, aunque el usuario simplemente hubiera pulsado el botón dos veces.
Solución	Cada publicación expone tres botones independientes: uno para dar o quitar like, uno para guardar o dejar de guardar, y una sección de comentarios con su propio campo de texto. Para likes y guardados, el servidor busca si ya existe un registro para ese usuario y esa publicación: si lo encuentra, lo borra; si no, lo crea. El like además notifica al autor, excepto cuando el autor se da like a sí mismo. Los comentarios siempre generan una entrada nueva; el borrado comprueba que quien borra sea el autor del comentario: si no lo es, el servidor no toca nada.
Motivación	El patrón de botón con toggle es el estándar en redes sociales: Instagram, Twitter y similares usan exactamente este modelo para likes y guardados. Como ventaja adicional, el frontend no necesita conocer el estado actual antes de llamar al servidor: simplemente envía la petición y refleja el resultado que recibe.
Cuestiones abiertas	Los comentarios no tienen paginación: si una publicación acumula muchos, todos se devuelven de golpe. No hay opción de editar un comentario ya publicado.
Alternativas	Endpoints separados por acción: un endpoint para dar like, otro para quitarlo, otro para guardar, otro para borrar guardado, etc. Más explícito, pero multiplica el número de endpoints y obliga al frontend a conocer el estado actual antes de cada llamada para saber cuál invocar.

Memorando técnico 013: Notificaciones con Menciones en Comentarios	
Asunto	Cada acción social genera una notificación para el destinatario correspondiente. Los comentarios añaden la posibilidad de mencionar a otros usuarios, que también deben ser notificados sin que nadie reciba dos notificaciones por el mismo evento.
Resumen	El servidor identifica al destinatario de cada acción y persiste la notificación. En comentarios, extrae los emails mencionados, carga los usuarios y lleva la cuenta de quién ya fue notificado para no repetir.
Factores causantes	Los nombres de usuario no son únicos, así que las menciones no pueden resolverse por nombre. Además, si el propietario de la publicación también aparece mencionado en el comentario, sin control recibiría dos notificaciones por el mismo evento.
Solución	El frontend codifica las menciones como <code>@[Nombre](email)</code> , usando el correo como identificador. Al publicar un comentario, el servidor extrae los emails mencionados en el texto, carga esos usuarios en una sola consulta y envía a cada uno una notificación, saltándose al propietario de la publicación. Para likes y follows, si el usuario da like, lo quita y lo vuelve a dar, solo la primera vez debe llegar una notificación al autor; las siguientes se descartan comprobando en base de datos si ya existe una notificación igual.
Motivación	Una sola consulta para todos los mencionados evita una consulta por cada mención. El conjunto de ya-notificados garantiza que nadie recibe dos notificaciones por el mismo comentario sin complicar el modelo de datos.
Cuestiones abiertas	Cualquier usuario puede escribir manualmente el patrón <code>@[Nombre](email)</code> en el campo de comentarios con el correo de cualquier persona registrada, y el servidor le enviará una notificación de mención sin que forme parte de la conversación.
Alternativas	Sin menciones: simplifica la implementación pero elimina una forma directa de dirigirse a otro usuario en un comentario. Se descartó porque los usuarios piloto solicitaron explícitamente esta funcionalidad en varias ocasiones durante las pruebas.

Memorando técnico 014: Centro de Notificaciones con Polling desde el Frontend	
Asunto	El usuario debe ver las notificaciones nuevas sin tener que recargar la página.
Resumen	La campana de notificaciones consulta periódicamente al servidor cuántas notificaciones sin leer hay y actualiza el contador (polling). Al abrir el panel, descarga la lista completa y marca todas como leídas.
Factores causantes	Las notificaciones las generan acciones de otros usuarios, por lo que el cliente no sabe cuándo llega una nueva. Sin consultas periódicas, el contador solo cambiaría al recargar la página.
Solución	La campana pregunta al servidor el número de no leídas cada cierto intervalo y actualiza el badge. Al hacer clic, descarga las notificaciones y las marca como leídas. Cada notificación puede eliminarse individualmente deslizándola. Si la notificación está relacionada con una publicación, al pulsar sobre ella el mapa navega directamente a esa publicación.
Motivación	El polling no requiere conexiones permanentes en el servidor y es suficiente para el volumen de usuarios esperado en Scubex. La latencia introducida por el intervalo de consulta es poco relevante en la práctica.
Cuestiones abiertas	El intervalo de polling fija el retraso máximo con el que el usuario ve una notificación nueva. No hay alertas en el sistema operativo cuando la pestaña está en segundo plano.
Alternativas	WebSockets: tiempo real completo, pero una mayor complejidad de infraestructura, innecesaria para la repercusión que tendría en la aplicación.

Cuadro 10.6: Memorando técnico 014: Centro de Notificaciones

Flujo de una interacción (like)

Cuando un usuario pulsa el botón de like en una publicación:

1. El usuario pulsa el botón. La interfaz actualiza el contador y el estado visual del botón de forma inmediata sin esperar respuesta del servidor (actualización optimista).
Al mismo tiempo, se envía la petición y el servidor comprueba si ya existe un like de ese usuario para esa publicación en base de datos.
2. Si ya existe, lo borra y devuelve al cliente el nuevo estado (sin like). No se genera ninguna notificación.
3. Si no existe, crea el registro y notifica al autor de la publicación. Si el usuario está dando like a su propia publicación, o ya había recibido una notificación de like del mismo actor, no se genera ninguna notificación adicional.
4. Si la respuesta del servidor no coincide con el estado optimista, la interfaz lo corrige para reflejar la realidad.

Nota — El flujo de guardados y el de follows siguen exactamente el mismo patrón de toggle. Las diferencias son dos: los guardados no generan ninguna notificación (es una acción privada del usuario), y el follow añade una comprobación extra que impide que un usuario se siga a sí mismo antes de llegar a la base de datos.

Flujo de un comentario con mención

Cuando un usuario publica un comentario que menciona a otro usuario:

1. El usuario envía el formulario. El frontend añade de inmediato un comentario temporal a la lista de comentarios del panel de detalle de publicación, incrementa el contador de comentarios visible en ese mismo panel y actualiza el badge del marcador en el mapa, todo sin esperar respuesta del servidor (actualización optimista).
La caja de texto se vacía para que el usuario pueda escribir otro comentario.
2. La petición llega al servidor y el comentario se persiste en base de datos con el texto tal como el usuario lo escribió. Las menciones viajan codificadas como `@[Nombre](email)`.

3. El servidor extrae los emails mencionados del texto y los resuelve a usuarios en una sola consulta a base de datos.
4. Si el comentarista no es el propietario de la publicación, se genera una notificación de tipo comentario para el propietario y se lo marca como ya notificado.
5. Por cada usuario mencionado que no haya sido ya notificado en el paso anterior, se genera una notificación de tipo mención. Esto evita que el propietario reciba dos notificaciones si también aparece mencionado.
6. El servidor devuelve el comentario real con el identificador asignado por la base de datos. El frontend sustituye el comentario temporal por el definitivo. Si la petición falló, deshace todos los cambios optimistas: elimina el comentario temporal, restaura el contador y devuelve el texto al campo de entrada.

Nota — Un comentario sin mención sigue exactamente el mismo camino en el frontend y en el servidor. La única diferencia es que la lista de emails mencionados queda vacía, por lo que el bucle de menciones no genera ninguna notificación adicional y el conjunto de ya-notificados no entra en juego.

10.4 DESPLIEGUE

Esta iteración añadió cinco nuevas tablas a la base de datos: `publication_likes`, `publication_saves`, `comments`, `user_follows` y `notifications`, todas creadas automáticamente por Hibernate al arrancar el servicio en Railway. No fue necesario añadir nuevas variables de entorno ni credenciales externas, ya que todas las funcionalidades de esta iteración utilizan exclusivamente la base de datos PostgreSQL existente.

10.5 PRUEBAS

Las pruebas cubren los tres desafíos técnicos documentados y verifican los pasos clave de los dos flujos.

Interacciones

- **Primer like** (Tabla §10.4, paso 3 del flujo de like): sin like previo, se persiste el registro y se notifica al autor.

- **Quitar like** (Tabla §10.4, paso 2 del flujo de like): si el like ya existe, se elimina y no se genera ninguna notificación.
- **Primer guardado y quitar guardado** (Tabla §10.4): mismo patrón que los likes, verificando que el registro se crea o se elimina según corresponda.
- **Añadir comentario** (Tabla §10.4, paso 1 del flujo de comentario): el comentario se persiste y se delega la notificación.
- **Borrar comentario por el autor** (Tabla §10.4): el autor borra su comentario; resultado verdadero y borrado ejecutado.
- **Borrar comentario por usuario ajeno** (Tabla §10.4): un usuario distinto al autor obtiene falso y no se ejecuta ningún borrado.
- **Borrar comentario inexistente** (Tabla §10.4): identificador no encontrado devuelve falso sin intentar borrar.

Notificaciones y menciones

- **Notificación de follow** (Tabla §10.5): seguir a otro usuario persiste una notificación de tipo seguimiento con el email del seguidor.
- **Auto-follow sin notificación** (Tabla §10.5): seguirse a uno mismo no genera ninguna notificación.
- **Deduplicación de follow y like** (Tabla §10.5): si ya existe una notificación del mismo actor para el mismo destinatario y tipo, no se persiste ninguna adicional.
- **Like propio** (Tabla §10.5): dar like a la propia publicación no genera notificación.
- **Comentario con mención** (Tabla §10.5, pasos 3–4 del flujo de comentario): un comentario con mención produce una notificación de comentario para el propietario y una de mención para el mencionado.
- **Mención al propietario sin duplicar** (Tabla §10.5, paso 4 del flujo de comentario): si el propietario es también el mencionado, solo se genera una notificación, no dos.
- **Texto largo truncado** (Tabla §10.5, paso 5 del flujo de comentario): un comentario de más de 150 caracteres produce un fragmento de exactamente 151 (150 + «...»).

- **Marcar como leída y eliminar: solo el destinatario** (Tabla §10.6): marcar como leída o eliminar una notificación ajena no persiste ningún cambio; las mismas operaciones sobre la propia notificación sí funcionan.

El proyecto cuenta con más pruebas unitarias (controladores de interacciones y notificaciones, endpoints de likes, guardados y comentarios) que no se detallan aquí por no estar directamente vinculadas a los desafíos técnicos documentados en esta sección.

PARTE IV

CIERRE DEL PROYECTO

MANUAL DE USUARIO

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

R esumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

11.1 SECCIÓN LIBRE

Estructurar en función del proyecto.

CONCLUSIONES

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

12.1 INFORME POST-MORTEM

Qué es un informe post-mortem

12.1.1 Lo que ha ido bien

- Argumento a favor 1.
- Argumento a favor 2.
- Argumento a favor 3.

12.1.2 Lo que ha ido mal

- Argumento en contra 1.
- Argumento en contra 2.
- Argumento en contra 3.

12.1.3 Discusión

En función de lo anterior, qué cambiaría si empezara hoy el proyecto de nuevo.

12.2 TRABAJOS FUTUROS

Enumera los puntos abiertos y que no se han resuelto. Indica si darían lugar a otro proyecto y de qué forma se podría acotar.

BIBLIOGRAFÍA